JA4proxy

Enterprise TLS Security Proxy

Technical Reference Manual

Complete reference for architects, developers, and security engineers

Version 2.0.0 | Last reviewed 2026-06-10 | github.com/seanpor/JA4proxy

POC — Functional & Tested

JA4proxy Documentation Series

Product Overview	Introduction for buyers and evaluators
Operator's User Guide	Installation, configuration, and day-to-day operations
Technical Reference Manual	**This document** — complete specification for all components

Status: POC — Functional and tested. Not yet production-hardened.

Source: `https://github.com/seanpor/JA4proxy` JA4 spec: `https://github.com/FoxIO-LLC/ja4`

Contents

List of Tables

System Architecture

JA4proxy makes allow/block/tarpit decisions based entirely on plaintext metadata visible before and during the TLS handshake. It never decrypts traffic, never holds TLS keys, and forwards allowed connections byte-for-byte unchanged.

Three design principles govern every architectural decision in JA4proxy: **fail open** (a missed threat is recoverable; a blocked legitimate user is not), **never decrypt** (the proxy is cryptographically transparent — it reads only the plaintext ClientHello), and **monitor first** (the default dial is 0; the system observes and scores before it ever blocks).

Design Principles

The Asymmetry Principle

The most important design constraint in JA4proxy is the cost asymmetry between the two error types:

Table 1: Error cost asymmetry — the governing principle of all threshold and TTL decisions

Error type	Example	Cost	Consequence
False negative	Bad bot slips through during cache sync window	Low	Recoverable. Log entry exists; retrospect analysis possible; no user harm.
False positive	Real browser is blocked or tarpitted	**High**	Not recoverable. User leaves. Revenue imp Reputation damage.

Every caching TTL, every threshold default, every fallback behaviour, and every new feature decision flows from this asymmetry. When the answer is not obvious, fail open.

The Asymmetry Rule

ALLOW decisions are cached for 30 minutes. BLOCK decisions are cached for 30 seconds. When Redis says block but the local in-process cache says allow, the local cache wins. If Redis is down, real browsers continue to work.

Never Decrypt

JA4proxy operates entirely on **plaintext metadata**. The TLS ClientHello — the first message a TLS client sends — is transmitted in plaintext before any encryption is established. It contains:

- The TLS version and list of supported cipher suites
- Supported extensions and their parameters
- The SNI (Server Name Indication) hostname
- ALPN (Application-Layer Protocol Negotiation) values
- Elliptic curve groups and point formats

From these fields, the JA4 fingerprinting algorithm produces a deterministic string that identifies the TLS client implementation (browser, bot, scanner, malware C2) with high accuracy. The proxy never participates in the TLS handshake — it is a passthrough observer and gatekeeper, not a terminator.

Monitor First

The default configuration ships with `dial: 0`. At dial=0, the pipeline runs completely — all signals are collected, all scores are computed, all decisions are made — but no connection is ever blocked. Every connection is allowed with a MONITOR annotation in the log.

This means operators can deploy JA4proxy in front of production traffic immediately, observe the score distribution, tune thresholds, and build confidence before enabling any blocking. Raising the dial is a conscious operator decision.

Config-Driven, Hot-Reloadable

Every behavioural parameter is configurable in `config/proxy.yml`. The proxy watches for `SIGHUP` and reloads configuration without dropping connections. The Management UI triggers reload via a Redis pub/sub message. After reload, new configuration applies to all subsequent connections.

Only the listen port, Redis URL, and TLS certificate paths require a process restart. All other parameters — dial, bypass conditions, thresholds, blacklists, whitelists — are hot-reloadable.[1]

[1] Config reload events are logged and reflected in the `ja4proxy_config_reloads_total` Prometheus counter.

Component Overview

Network Topology

Traffic Flow Diagram

Network Zones

A production deployment uses four network zones:

Table 2: System components and their responsibilities

Component	Language	Port	Responsibility
HAProxy	C	:443	TLS passthrough load balancer; PROXY protocol v2 injection; upstream TLS listener
JA4proxy Proxy	Go*	:8080	TLS ClientHello sniffing; pipeline execution; allow/block/tarpit/ban decisions; Redis I/O
Redis	Redis 7	:6379	Shared state: bans, rate windows, beaconing timestamps, JA4 lists, HLL per CIDR, event stream
Analytics Node	Python	—	Reads Redis Streams (`XREADGROUP`); cross-instance aggregation; campaign detection; writes findings to Redis
Prometheus	Go	:9090	Metrics collection and storage
Grafana	Go	:3001	Dashboards; visualisation of proxy telemetry
Loki	Go	:3100	Structured log aggregation
Alertmanager	Go	:9093	Alert routing and deduplication
Backend	any	:443	Protected origin service; receives only allowed, unmodified TLS connections

*The Go proxy (`cmd/proxy/`, `internal/`) was promoted to production in Phase 15 and is the only implementation that ships in releases, Docker images, and Helm charts. The Python proxy (`proxy.py`, `src/security/`) is retained as an experimental prototyping surface for new signal modules; production-runtime Python is limited to the Management API, analytics node, and compliance reporting — services that are not on the proxy hot path.

The Backend origin server sits behind the App zone, reachable only from JA4proxy instances. Internet traffic never reaches the App zone directly. The Data zone is not routable from the DMZ or from the internet.

Data Flow: TLS Connection Lifecycle

Stage 1 — TCP Accept and IP Extraction

When HAProxy accepts a TLS connection, it wraps it in a PROXY protocol v2 header[2] and forwards the raw TCP stream to a JA4proxy instance on port 8080.

[2] PROXY protocol v2 is a binary framing format prepended by HAProxy to the TCP stream. It carries the original client IP:port and the HAProxy IP:port.

JA4proxy reads the PROXY protocol header first, extracting:

- The **real client IP** (IPv4 or IPv6)
- The real client port
- The HAProxy (sender) IP — used for upstream CIDR trust verification

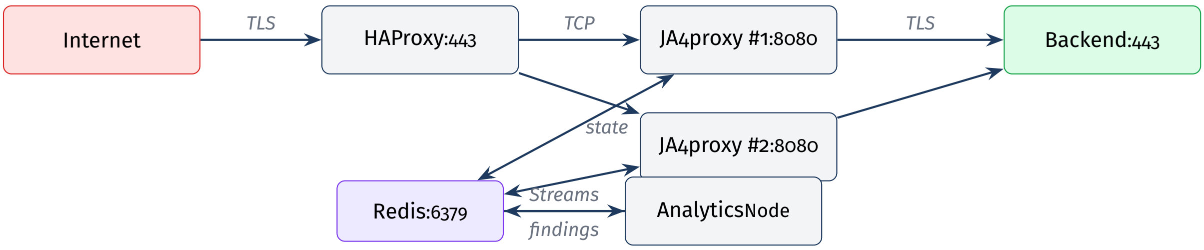

Figure 1: HAProxy headers so client IP.

Table 3: Network zone definitions and permitted traffic

Zone	Subnet (example)	Contains	Permitted inbound
DMZ	10.0.1.0/24	HAProxy load balancer	Internet :443 only
App	10.0.2.0/24	JA4proxy proxy instances	HAProxy TCP from DMZ only
Data	10.0.3.0/24	Redis, analytics node	App zone on :6379 only
Management	10.0.4.0/24	Prometheus, Grafana, Loki	Operator VPN only

The client IP is canonicalised to its compressed string representation (`ipaddress.ip_address(ip).compressed` in Python, `netip.Addr.String()` in Go) immediately. All subsequent pipeline steps use this canonical form.

Stage 2 — TLS ClientHello Sniff

After PROXY protocol extraction, JA4proxy reads the first TLS record from the TCP stream. This is always the ClientHello — a plaintext record (record type 0x16, content type 0x01). The proxy parses:

- Handshake type and TLS record version
- Client random (32 bytes, not used for fingerprinting)
- Session ID length and value
- Cipher suite list (ordered)
- Compression methods
- Extensions: SNI (0x0000), ALPN (0x0010), supported groups (0x000a), EC point formats (0x000b), signature algorithms (0x000d), supported versions (0x002b), PSK key exchange modes (0x002d), key share (0x0033), and others

From this data, the JA4, JA4X, and JA4T fingerprints are computed (see Chapter). The entire sniff operation is non-blocking and completes before any response is sent.

Stage 3 — Pipeline Execution

The pipeline (Chapter) runs all 12 layers. Layers 0–2 are bypass/block decisions that short-circuit the pipeline immediately. Layers 4–8 are signals that run in parallel via `asyncio.gather()`. The pipeline produces a score (0–100) and an action.

Stage 4 — Action Execution

Based on the action from the pipeline:

ALLOW The buffered ClientHello bytes are forwarded to the backend. JA4proxy acts as a transparent TCP relay for the remainder of the connection.

BLOCK A TCP RST is sent. The connection is closed immediately.

TARPIT The connection is accepted but data is delivered to the backend at an artificially low rate (or deliberately delayed), consuming attacker resources.

BAN The IP is added to Redis with a TTL. Subsequent connections from this IP are rejected immediately at layer 0 without pipeline execution.

Flag The connection is allowed but a high-priority alert is emitted to the Redis Stream for the Analytics node.

Stage 5 — Event Emission

Every connection that completes pipeline evaluation produces a JSON event written to the `events` Redis Stream via `XADD` (fire-and-forget). The analytics node consumes these events for cross-instance aggregation, campaign detection, and score drift alerting. Stream writes add zero latency to the hot path.

IPv6 Handling

IPv6 is a first-class concern throughout JA4proxy. Every feature that touches an IP address handles both address families:

Table 4: IPv6 handling rules by subsystem

Subsystem	IPv4 rule	IPv6 rule
Rate limiting keys	Full IP string	Full IP string (no truncation)
CIDR matching	In-process pytricia / Go trie	Same trie handles both natively
GeoIP lookup	MaxMind GeoLite2 City/ASN	Same DB covers IPv6
HyperLogLog per subnet	`hll:cidr24:{cidr}` (/24)	`hll:cidr48:{cidr}` (/48)
Beaconing key	`beacon:{ip}:{ja4}`	Same key format, full IPv6
Ban key	`ban:{ip}`	Same key format, full IPv6
RDAP lookup	RIR via IANA bootstrap	Same bootstrap — works for IPv6
Block expansion	Never beyond /24	Never beyond /48

IPv6 /48 Equivalence

A /48 IPv6 prefix is the approximate equivalent of an IPv4 /24 for population density purposes. A typical ISP assigns a /48 to a single customer premises. Block expansion (RDAP Phase 11) never expands beyond /48 for IPv6 or /24 for IPv4 — expanding further would risk blocking entire ISP customer populations.

The Go and Python Implementations

Go (Production Runtime)

The Go proxy in `cmd/proxy/` and `internal/` (built to `bin/proxy`) is the production runtime. It was promoted to production in Phase 15 and is the only implementation that ships in releases, Docker images, Helm charts, and enterprise documentation. The Go implementation uses `net/netip` for IP handling, a `crypto/tls`-derived ClientHello parser, and goroutine-per-connection concurrency. It targets 10–50× the throughput of the Python prototype.

Python (Experimental Prototyping Surface)

The Python implementation in `proxy.py` and `src/security/` is retained as an experimental prototyping surface for new signal modules. Engineers prototype in Python, prove the idea, then port it to Go before it ever touches real traffic. Python services that are *not* the proxy — the Management API (FastAPI), the analytics node, and the compliance reporting service — remain production Python because they are not on the connection-handling hot path.

Implementation Status

> The Go proxy is the only path that handles production traffic. The Phase 200-series hardening (Redis TLS, PROXY v2 with TLVs, default-credential removal, expanded signal coverage) lands in the Go runtime first; Python is updated only when needed for prototyping.

Horizontal Scaling

Multiple JA4proxy instances share state through Redis. Because all ban decisions, rate windows, JA4 lists, and beaconing data live in Redis, any instance can enforce decisions made by any other instance. There is no sticky routing requirement — HAProxy distributes connections across instances with standard round-robin or least-connection load balancing.

The only per-instance state is the in-process LRU cache (`src/cache/local_cache.py`). This cache is intentionally not synchronised across instances. Cache misses are handled by Redis fallback. The local cache uses conservative TTLs (30min for ALLOW, 30s for BLOCK) to bound the window in which a newly added ban is not yet enforced by a particular instance.

The Security Pipeline

The pipeline is the ordered sequence of checks applied to every TLS connection. Every layer runs in microseconds. Signal collection layers run in parallel. The pipeline produces exactly one action per connection.

The pipeline has three zones: the **fast-path bypass zone** (layers 0–2), which short-circuits immediately on certain conditions; the **signal collection zone** (layers 3–10), where eight checks run concurrently; and the **decision zone** (layers 11–12), which aggregates signals into a score and maps the score to an action.

Pipeline Overview

Table 5: Complete pipeline layer reference

Layer	Name	Zone	Output	Trigger condition
0	IP trust & normalisation	Fast path	—	Every connection
0b	Static IP allowlist	Fast path	ALLOW	IP in static allowlist
0c	GeoIP country block	Fast path	BLOCK	Country in blacklist
0d	CIDR block	Fast path	BLOCK	IP in blocked subnet
0e	Spamhaus DROP/EDROP	Fast path	BLOCK	IP in DROP/EDROP trie
1	ALPN browser bypass	Fast path	ALLOW	ALPN = h2 or h1
1b	JA4 whitelist	Fast path	ALLOW	JA4 in whitelist SET
1c	mTLS client cert	Fast path	ALLOW	Valid client certificate
2	JA4 blacklist	Fast path	BLOCK	JA4 in blacklist SET
3	TLS enforcement	Signal	Signal or BLOCK	TLS version/cipher check
4	SNI analysis	Signal	Signal	Every connection
5	TCP & connection behaviour	Signal	Signal	Every connection
6	ASN classification	Signal	Signal	Every connection
7	FCrDNS enrichment	Signal	Signal	Every connection
8	Beaconing detector	Signal	Signal	Every connection
9	AbuseIPDB score	Signal	Signal	Every connection
10	RDAP org reputation	Signal	Signal	Every connection
11	Attacker attribution	Analysis	Signal	Cross-IP JA4/JA4X correlation
12	Behavioural analysis	Analysis	Signal	Sequential/coordinated patterns
13	Risk scorer	Decision	Score 0–100	Always
14	Action decider	Decision	Action	Always

The Fast-Path Bypass System

How Bypasses Work

Each bypass condition is independently configurable under the `security_policy` configuration section. When a bypass condition is met and its bypass is enabled, the pipeline short-circuits immediately — the signal collection and scoring layers never execute, and no Redis signal-collection writes are made.

All bypass conditions are toggleable. Disabling an ALLOW bypass causes those connections to pass through the full scorer. Disabling a BLOCK bypass routes those connections through the scorer, where the dial controls whether they are actually blocked.

Bypass Modification Risk

The proxy emits a startup `WARN` log entry and increments a Prometheus counter for every high-risk bypass that is disabled. Operators should monitor `ja4proxy_bypass_total` to verify bypass layer traffic volumes are as expected.

Layer 0 — IP Trust and Normalisation

Every connection enters layer 0. This layer:

1. Verifies the connection originates from a trusted upstream CIDR (the HAProxy address range)
2. Reads the PROXY protocol v2 header to extract the real client IP
3. Canonicalises the IP to its compressed string representation
4. Checks whether the IP is in the in-process ban cache (fast reject before Redis)

If the sender is not in the trusted upstream CIDR, the connection is rejected immediately. This prevents IP spoofing via injected PROXY protocol headers.[3]

[3] Without trusted upstream CIDR verification, an attacker who can reach port 8080 directly could inject a PROXY header claiming any client IP.

Layer 0b — Static IP Allowlist

Configuration key: `security_policy.static_ip_allowlist.enabled`

If the client IP is found in the static IP allowlist (configured in `config/proxy.yml` and hot-reloadable), the connection is immediately allowed without scoring. Typical use: trusted monitoring systems, CI test runners, internal health checkers.

Layer 0c — GeoIP Country Block

Configuration key: `security_policy.country_blacklist_bypass.enabled`

Country lookup uses the IP2Location LITE database in conjunction with a Redis-backed country blacklist. Countries in `geoip.country_` blacklist trigger an immediate BLOCK. This check runs before any TLS parsing — the country decision is based on IP alone.

Country Allowlist vs Blacklist

geoip.country_whitelist_enabled and geoip.country_blacklist_enabled are independent toggles. The allowlist mode (only listed countries are permitted) is distinct from the blacklist mode (listed countries are blocked). Both can be active simultaneously; the whitelist check runs first.

Layer 0d — CIDR Block

Subnet blocks are maintained in Redis and refreshed every 30 seconds into an in-process trie. This layer is the enforcement point for RDAP block expansion (Phase 11) and manually added subnet bans. Matching an entry produces an immediate BLOCK without scoring.

IPv6: The trie handles both IPv4 and IPv6 prefixes natively. Block expansion never adds prefixes shorter than /24 (IPv4) or /48 (IPv6).

Layer 0e — Spamhaus DROP/EDROP

Configuration key: `security_policy.spamhaus_bypass.enabled`

Spamhaus DROP and EDROP lists are fetched at startup and refreshed every `spamhaus.refresh_interval` seconds (default 3600). They are stored in an in-process trie for O(log n) lookup.[4]

[4] The Spamhaus DROP list contains IP ranges that have been hijacked or directly allocated to cybercriminals. False positive rate is near zero — these ranges are never used for legitimate traffic.

When disabled, Spamhaus matches produce a `RiskSignal(+80)` instead of a hard block. The signal flows through the scorer, and the dial controls whether the connection is blocked.

Layer 1 — ALPN Browser Bypass

Configuration key: `security_policy.alpn_browser_bypass.enabled`

ALPN Bypass — Do Not Disable Without Justification

The ALPN browser bypass is structurally the most critical bypass. h2 and h1 ALPN values are set exclusively by real browsers communicating over HTTPS. Disabling this bypass causes all browser traffic to be scored. With typical score distributions, this will generate false positives at scale. This bypass should only be disabled to score specific h2 API clients, and only with monitoring in place.

Connections presenting `h2` or `h1` ALPN values are immediately allowed. These connections are never enqueued for enrichment, and their IP/netblock is never eligible for block expansion.

Layer 1b — JA4 Whitelist

Configuration key: `security_policy.ja4_whitelist_bypass.enabled`

The JA4 whitelist is a Redis SET (`ja4:whitelist`) loaded into an in-process set at startup and refreshed on config reload. JA4 fingerprints matching the whitelist trigger an immediate ALLOW. Wildcard and regex matching are not supported — whitelist entries are exact fingerprint strings.

Layer 1c — mTLS Client Certificate

Configuration key: `security_policy.mtls_bypass.enabled`

A valid mTLS client certificate is cryptographic proof of identity. By default, presenting a valid certificate against the configured trusted CA set (`config/trusted_cas.pem`) triggers an immediate ALLOW without scoring. If disabled, mTLS clients are scored — they can still be blocked if their score exceeds the dial threshold.

Layer 2 — JA4 Blacklist

Configuration key: `security_policy.ja4_blacklist_bypass.enabled`

The JA4 blacklist is a Redis SET (`ja4:blacklist`) mirroring the whitelist architecture. Matching fingerprints trigger an immediate TCP RST with no further processing. Known blacklist entries include tool and C2 implant fingerprints such as `t13d190900_9dc949149365_97f8aa674fd9` (Sliver C2 framework).

When disabled, blacklisted fingerprints produce a high-weight `RiskSignal` routed through the scorer. This is useful for investigating traffic from known-bad fingerprints without hard-blocking.

Signal Collection (Layers 3–10)

Layers 3–10 run concurrently via `asyncio.gather()` after the fast-path zone completes without a decision. Each signal check is a coroutine that:

1. Reads relevant data from the connection context and Redis
2. Optionally calls an external service (AbuseIPDB, RDAP, DNS)
3. Returns a `RiskSignal(score, weight, reason)` object

All signal checks are non-blocking. External service calls are fire-and-forget via `asyncio.create_task()` — they do not block the connection handling coroutine. Every signal check has an explicit failure handler that logs the failure, increments a Prometheus error counter, and returns a zero/neutral result.

See Chapter for full documentation of each signal.

Attacker Attribution (Layer 11)

Layer 11 correlates JA4, JA4X, and JA4T fingerprints across IPs observed by the analytics node. It looks for:

- Multiple IPs sharing the same JA4/JA4T profile (coordinated scanning)
- JA4X extension ordering inconsistencies with claimed browser identity
- Fingerprint drift — the same IP changing fingerprints across connections

Attribution findings are written by the analytics node to `findings:{ip}` in Redis (TTL=300s) and read back by the pipeline at layer 11.

Behavioural Analysis (Layer 12)

Layer 12 detects behavioural patterns that individual connections cannot reveal:

- **Sequential probing** — connections to incrementally varying SNI values from the same source IP, characteristic of host enumeration
- **Coordinated bursts** — sudden increases in connection rate from multiple IPs sharing a fingerprint (botnet activation pattern)
- **Fingerprint drift** — the same IP cycling through multiple JA4 values to evade per-fingerprint rate limits

Risk Scorer (Layer 13)

Score Aggregation Algorithm

The risk scorer aggregates all `RiskSignal` objects from layers 3–12 into a single integer score in the range 0–100. The aggregation algorithm is confidence-weighted:

1. Each signal contributes `signal.score` × `signal.weight` to the total
2. Weights are normalised to sum to 1.0 across all active signals
3. The raw weighted sum is clamped to [0, 100]
4. Signal scores from external services that are unreachable contribute 0 (fail open — a missing signal never increases the score)

Score Semantics

Table 6: Score ranges and their interpretation

Score	Interpretation	Typical source	Default action at dfal=100
0–19	Clean	Browser traffic, known-good fingerprints	Allow
20–39	Low risk	Datacenter IP with clean fingerprint	Allow or flag
40–59	Moderate risk	Scanner, unresolved PTR, high-confidence AbuseIPDB	Rate limit
60–79	High risk	Tor exit, known-bad ASN, beaconing pattern	Tarpit
80–100	Critical	Confirmed C2 fingerprint, Spamhaus signal, banned IP	Block or ban

Action Decider (Layer 14)

The Dial

The `dial` value (0–100) controls how aggressively scores are translated into blocking actions. At `dial=0`, every connection is allowed regardless of score — the system is in pure monitor mode. At `dial=100`, the thresholds configured in `security.rate_limit_strategies` apply in full.

Dial Increment Policy

The dial should be raised incrementally. A jump from 0 to 100 without first validating the score distribution risks generating false positives at scale. The recommended approach is to raise the dial to 20 and observe for 24 hours, then to 50, then to 100.

The Six Actions

Table 7: Action definitions and connection handling behaviour

Action	Badge	Connection fate	Notes
allow	ALLOW	Forwarded to backend unchanged	ClientHello bytes replayed to backend; proxy becomes transparent relay
flag	ALLOW	Forwarded to backend	Connection allowed but high-priority event emitted to analytics stream
rate_limit	MONITOR	Queued or delayed	Token bucket or sliding window rate limiting applied per-IP or per-JA4
tarpit	TARPIT	Artificially slowed	Data delivered to backend at throttled rate; consumes attacker connection slots
block	BLOCK	TCP RST sent	Connection closed immediately; no data forwarded; TTL-free (not persistent)
ban	BAN	TCP RST sent	IP added to Redis ban SET with TTL; all future connections blocked immediately at layer 0

Ban Escalation

Ban escalation applies when an IP continues to trigger blocks after an initial ban expires. The ban TTL doubles on each re-ban up to the configured maximum:

- First ban: `ban_duration` seconds (default 300)
- Second ban: `ban_duration` $\times$ 2 seconds
- Subsequent bans: capped at 604800 seconds (7 days)

Ban state is stored in Redis as `ban:{ip}` with the TTL set at write time. The proxy reads ban state from its local in-process cache first (TTL=30s) to avoid Redis round-trips for known-banned IPs.

Multi-Strategy Policy

Three rate-limiting strategies run simultaneously, each tracking a different key:

by_ip Rate window keyed by client IP. Detects high-volume floods from a single address.

by_ja4 Rate window keyed by JA4 fingerprint. Detects distributed scanning sharing a tool fingerprint.

by_ip_ja4_pair Rate window keyed by (IP, JA4) pair. Detects IP-rotation evasion where each IP presents the same fingerprint.

The `security.multi_strategy_policy` setting controls how strategy results are combined:

`any` Block if *any* strategy triggers (most aggressive)

`majority` Block if *two or more* strategies trigger (default)

`all` Block only if *all* strategies trigger (most conservative)

Pipeline Performance Characteristics

Table 8: Expected latency contribution by pipeline zone

Zone	**Python (asyncio)**	**Go**	**Notes**
Fast-path bypass (cache hit)	$<100\,\mu s$	$<10\,\mu s$	Local LRU cache, no Redis
Fast-path bypass (Redis)	200–400 μs	150–300 μs	Single Redis GET
Signal collection (parallel)	1–5 ms	0.1–1 ms	External services: AbuseIPDB 50ms (cached)
Scoring and decision	$<100\,\mu s$	$<10\,\mu s$	Pure computation
Total (cached, fast path)	$<500\,\mu s$	$<50\,\mu s$	Typical cached connection
Total (full pipeline, no cache)	2–10 ms	0.3–2 ms	First-seen IP, all signals

AbuseIPDB Latency

AbuseIPDB lookups use aggressive caching (TTL=3600s by default). A cache hit adds zero latency. Only cache misses (first-seen IP) incur the network round-trip, which is 50ms and runs asynchronously — it does not block the connection decision.

Configuration Reference

Every behavioural parameter in JA4proxy is configurable in `config/proxy.yml`. All keys except listen port, Redis URL, and TLS certificate paths support hot-reload via `SIGHUP` or the Management UI.

The configuration file is `config/proxy.yml`. Environment variable interpolation is supported: `${VAR_NAME}` is replaced with the environment variable at startup. Secrets (API keys, Redis password) must be provided via environment variables — never hard-coded in the config file.

Hot Reload Behaviour

On `SIGHUP`, JA4proxy re-reads `config/proxy.yml` and applies all changes that support hot-reload. A `config_reload` event is emitted to the policy audit log in Redis. The Management UI sends a Redis pub/sub message to trigger reload without a Unix signal.

Table 9: Hot-reload support by configuration section

Section	**Hot-reloadable**	**Notes**
`proxy.bind_host` / `bind_port`	No	Requires process restart
`proxy.backend_host` / `backend_port`	Yes	Applies to new connections
`dial`	Yes	Applies immediately to all new decisions
`security_policy`	Yes	New bypass states apply to next connection
`geoip`	Yes	New country lists apply immediately
`security` (lists)	Yes	List changes apply to next connection
`abuseipdb`	Yes	Cache TTL change affects new lookups
`rdap`	Yes	block_expansion toggle applies immediately
`spamhaus`	No (refresh_interval)	Next trie refresh uses new URL
`redis.url` / `redis.password`	No	Requires process restart

proxy — Core Listener Settings

```yaml
proxy:
  bind_host: "0.0.0.0"
  bind_port: 8080
  backend_host: "backend"   # Docker service name or hostname
```

Table 10: `proxy` section — core listener settings

Key	Type	Default	Description
`bind_host`	string	`"0.0.0.0"`	Listen address for incoming TCP connections from HAProxy
`bind_port`	integer	`8080`	Listen port. Must match HAProxy backend configuration
`backend_host`	string	*required*	Hostname or IP of the protected backend. Must be set
`backend_port`	integer	`443`	Backend port. Typically 443 for HTTPS

```
  backend_port: 443
```

dial — Blocking Aggression

Table 11: `dial` setting

Key	Type	Default	Description
`dial`	integer	`0`	Blocking aggression: 0 = monitor only (no blocking); 100 = full blocking per configured thresholds. Intermediate values scale blocking proportionally. Raise incrementally.

Never Deploy with dial=100 on First Install

Always start at `dial: 0`. Observe the score distribution for at least 24 hours before raising the dial. A premature jump to 100 can block legitimate traffic before you have validated your threshold configuration.

security_policy — Bypass Conditions

Each bypass condition has a single `enabled` boolean key. All default to `true`.

```yaml
security_policy:
  alpn_browser_bypass:
    enabled: true    # h2/h1 ALPN -> ALLOW without scoring
  ja4_whitelist_bypass:
    enabled: true    # JA4 in whitelist -> ALLOW without scoring
  mtls_bypass:
    enabled: true    # Valid mTLS cert -> ALLOW without scoring
  static_ip_allowlist:
    enabled: true    # IP in static allowlist -> ALLOW without scoring
  ja4_blacklist_bypass:
    enabled: true    # JA4 in blacklist -> immediate RST, no scoring
  country_blacklist_bypass:
    enabled: true    # Country in GeoIP blocklist -> immediate block
  spamhaus_bypass:
    enabled: true    # Spamhaus DROP/EDROP -> immediate block
```

Table 12: security_policy section — bypass conditions

Key	Default	Effect when true	Risk if disabled
`alpn_browser_bypass.enabled`	`true`	h2/h1 ALPN → ALLOW without scoring	Browser traffic scored; false positive risk elevated
`ja4_whitelist_bypass.enabled`	`true`	JA4 in whitelist → ALLOW without scoring	Whitelisted fingerprints scored; may be blocked if above threshold
`mtls_bypass.enabled`	`true`	Valid mTLS cert → ALLOW without scoring	mTLS clients scored; may be blocked if above threshold
`static_ip_allowlist.enabled`	`true`	IP in allowlist → ALLOW without scoring	Allowlisted IPs scored
`ja4_blacklist_bypass.enabled`	`true`	JA4 in blacklist → TCP RST immediately	Blacklisted fingerprints only blocked if score exceeds dial threshold
`country_blacklist_bypass.enabled`	`true`	Country in blacklist → BLOCK immediately	Blacklisted countries only blocked by scorer
`spamhaus_bypass.enabled`	`true`	Spamhaus DROP match → BLOCK immediately	Spamhaus matches produce RiskSignal(+80) instead of hard block
`tls_version_bypass.enabled`	`true`	TLS 1.0/1.1/SSLv3 → TCP RST immediately	Old TLS versions produce risk score contribution instead of hard reject

```yaml
tls_version_bypass:
  enabled: true    # TLS 1.0/1.1/SSLv3 -> immediate RST
```

geoip — Geographic Controls

Table 13: geoip section

Key	Type	Default	Description
`country_whitelist_enabled`	boolean	`true`	Enable country allowlist mode. If true, only countries in `country_whitelist` are allowed
`country_whitelist`	list	`["IE","GB","IM","US"]`	ISO 3166-1 alpha-2 country codes permitted when whitelist mode is active
`country_blacklist_enabled`	boolean	`true`	Enable country blacklist mode. Listed countries are immediately blocked
`country_blacklist`	list	`["KP","RU"]`	ISO 3166-1 alpha-2 country codes to block immediately

Whitelist and Blacklist Together

Both modes can be active simultaneously. The whitelist check runs first. A country not on the whitelist (when whitelist mode is active) is blocked independently of whether it appears on the blacklist. Use whitelist mode for strict geographic restrictions (allowlist your expected countries), and blacklist mode for blocking specific known-bad countries when you serve a broad geographic audience.

```yaml
geoip:
  country_whitelist_enabled: true
  country_whitelist: ["IE", "GB", "IM", "US"]
  country_blacklist_enabled: true
  country_blacklist: ["KP", "RU"]
```

security — Fingerprint Lists and Rate Limits

Fingerprint Lists

Table 14: `security` section — fingerprint lists

Key	Type	Default	Description
`whitelist`	list of strings	`[]`	Exact JA4 fingerprints to whitelist. Loaded into Redis `ja4:whitelist` and an in-process set
`whitelist_patterns`	list of strings	`["h2","h1"]`	ALPN patterns that trigger the ALPN browser bypass. Separate from the JA4 whitelist
`blacklist`	list of strings	`[]`	Exact JA4 fingerprints to blacklist. Loaded into Redis `ja4:blacklist`
`fingerprint_labels`	map	`{}`	Human-readable names for fingerprints. Used in logs and metrics
`multi_strategy_policy`	string	`"majority"`	How to combine rate limit strategy results: `any` / `majority` / `all`

Rate Limit Strategies

Three rate-limiting strategies are defined under `security.rate_limit_strategies`. Each has an identical structure:

Table 15: `rate_limit_strategies` — per-strategy keys

Key	Type	Default	Description
`thresholds.suspicious`	integer	—	Connections/second at which the strategy flags as suspicious
`thresholds.block`	integer	—	Connections/second at which the strategy triggers a block
`thresholds.ban`	integer	—	Connections/second at which the strategy triggers a permanent ban
`action`	string	—	Action to take when threshold is crossed: `tarpit` or `block`
`ban_duration`	integer	`300`	Initial ban TTL in seconds. Escalates on re-ban

```yaml
security:
  whitelist:
    - "t13d1516h2_8daaf6152771_02713d6af862"  # Chrome 120+
  whitelist_patterns:
    - "h2"
    - "h1"
  blacklist:
    - "t13d190900_9dc949149365_97f8aa674fd9"  # Sliver C2
```

```
  fingerprint_labels:
    "t13d1516h2_8daaf6152771_02713d6af862": "Chrome 120+"
  multi_strategy_policy: "majority"
  rate_limit_strategies:
    by_ip:
      thresholds: {suspicious: 50, block: 200, ban: 500}
      action: "tarpit"
      ban_duration: 300
    by_ja4:
      thresholds: {suspicious: 20, block: 100, ban: 200}
      action: "block"
      ban_duration: 300
    by_ip_ja4_pair:
      thresholds: {suspicious: 20, block: 50, ban: 100}
      action: "tarpit"
      ban_duration: 300
```

abuseipdb — Reputation Lookup

Table 16: `abuseipdb` section

Key	Type	Default	Description
`enabled`	boolean	`true`	Enable AbuseIPDB lookups
`api_key`	string	*required*	AbuseIPDB API key. Must be an env var reference: `${ABUSEIPDB_API_KEY}`
`cache_ttl`	integer	`3600`	Seconds to cache AbuseIPDB responses in Redis
`min_score_to_signal`	integer	`50`	Minimum AbuseIPDB score (0–100) that produces a RiskSignal. Never hard-block below 50

AbuseIPDB Score 50 Floor

The `min_score_to_signal` floor exists because AbuseIPDB scores below 50 are common for shared infrastructure (NAT gateways, VPNs, corporate egress). Hard-blocking based on scores below 50 has an unacceptably high false positive rate. The default of 50 should not be lowered without evaluating the false positive impact on your traffic.

```
abuseipdb:
  enabled: true
  api_key: "${ABUSEIPDB_API_KEY}"
  cache_ttl: 3600
  min_score_to_signal: 50
```

Table 17: rdap section

Key	Type	Default	Description
enabled	boolean	true	Enable RDAP/WHOIS enrichment lookups
block_expansion	boolean	false	Automatically block the entire /24 (IPv4) or /48 (IPv6) of a blocked IP. Must be explicitly enabled
max_expansion	integer	24	Maximum prefix length for block expansion. Never set below 24 for IPv4

rdap — WHOIS/RDAP Org Reputation

Block Expansion Off by Default

`block_expansion: true` will cause JA4proxy to block the entire /24 subnet when a single IP is blocked. On residential ISPs and shared hosting, an entire /24 may serve thousands of unrelated users. Enable only after confirming that the blocked IPs are from dedicated attacker infrastructure with no residential or business customers.

```yaml
rdap:
  enabled: true
  block_expansion: false      # secops must explicitly opt in
  max_expansion: 24           # never beyond /24 IPv4, /48 IPv6
```

spamhaus — DROP/EDROP Blocklists

Table 18: spamhaus section

Key	Type	Default	Description
drop_list_url	string	https://www.spamhaus.org/drop/drop.txt	URL to the Spamhaus DROP list
edrop_list_url	string	https://www.spamhaus.org/drop/edrop.txt	URL to the Spamhaus EDROP list
refresh_interval	integer	3600	Seconds between trie refresh cycles

```yaml
spamhaus:
  drop_list_url: "https://www.spamhaus.org/drop/drop.txt"
  edrop_list_url: "https://www.spamhaus.org/drop/edrop.txt"
  refresh_interval: 3600
```

redis — Connection Settings

Table 19: redis section

Key	Type	Default	Description
url	string	"redis://redis:6379"	Redis connection URL. For Unix socket: unix:///var/run/redis/redis.sock
password	string	*optional*	Redis AUTH password. Use env var: ${REDIS_PASSWORD}

Unix Domain Socket Performance

For deployments where JA4proxy and Redis are on the same host, using a Unix domain socket for the Redis URL reduces latency by approximately 30–40% compared to TCP loopback. Set `url: "unix:///var/run/redis/redis.sock"` and ensure the socket path is accessible inside the container.

```yaml
redis:
  url: "redis://redis:6379"
  password: "${REDIS_PASSWORD}"
```

Complete Configuration Example

```yaml
# config/proxy.yml - complete reference configuration

proxy:
  bind_host: "0.0.0.0"
  bind_port: 8080
  backend_host: "backend"
  backend_port: 443

# 0 = monitor only. Raise incrementally after validating score
    distribution.
dial: 0

security_policy:
  alpn_browser_bypass:
    enabled: true
  ja4_whitelist_bypass:
    enabled: true
  mtls_bypass:
    enabled: true
  static_ip_allowlist:
    enabled: true
  ja4_blacklist_bypass:
    enabled: true
  country_blacklist_bypass:
    enabled: true
  spamhaus_bypass:
    enabled: true
  tls_version_bypass:
    enabled: true

geoip:
  country_whitelist_enabled: true
  country_whitelist: ["IE", "GB", "IM", "US"]
  country_blacklist_enabled: true
  country_blacklist: ["KP", "RU"]

security:
  whitelist:
    - "t13d1516h2_8daaf6152771_02713d6af862"  # Chrome 120+
```

```
  whitelist_patterns:
    - "h2"
    - "h1"
  blacklist:
    - "t13d190900_9dc949149365_97f8aa674fd9"  # Sliver C2
  fingerprint_labels:
    "t13d1516h2_8daaf6152771_02713d6af862": "Chrome 120+"
  multi_strategy_policy: "majority"
  rate_limit_strategies:
    by_ip:
      thresholds: {suspicious: 50, block: 200, ban: 500}
      action: "tarpit"
      ban_duration: 300
    by_ja4:
      thresholds: {suspicious: 20, block: 100, ban: 200}
      action: "block"
      ban_duration: 300
    by_ip_ja4_pair:
      thresholds: {suspicious: 20, block: 50, ban: 100}
      action: "tarpit"
      ban_duration: 300

abuseipdb:
  enabled: true
  api_key: "${ABUSEIPDB_API_KEY}"
  cache_ttl: 3600
  min_score_to_signal: 50

rdap:
  enabled: true
  block_expansion: false
  max_expansion: 24

spamhaus:
  drop_list_url: "https://www.spamhaus.org/drop/drop.txt"
  edrop_list_url: "https://www.spamhaus.org/drop/edrop.txt"
  refresh_interval: 3600

redis:
  url: "redis://redis:6379"
  password: "${REDIS_PASSWORD}"
```

Signal Reference

A signal is a scored observation. Each of the eight signal checks contributes a score in [0, 100] and a confidence weight. The risk scorer combines them. No single signal can block a connection — only the combined score, acting through the dial, produces blocking actions.

All eight signals run concurrently after the fast-path bypass zone. Each signal is independent. An unreachable external service contributes zero — it never increases the score (fail open). Signal results are logged in the `signals` field of every connection event.

Signal Score Model

Each signal produces a `RiskSignal` with three fields:

`score` Integer 0–100. Higher means more suspicious.

`weight` Float 0.0–1.0. The confidence weight of this signal type. High-confidence signals (TLS version, JA4 blacklist category) have higher weights. Low-certainty signals (RDAP org, passive DNS) have lower weights.

`reason` Human-readable explanation for the log.

The risk scorer computes: final_score $= \text{clamp}(\sum_i \text{score}_i \times \text{weight}_i, 0, 100)$ where weights are normalised to sum to 1.0 across all active signals.

Table 20: Signal summary — all eight signals

#	Signal	Phase	Default weight	External service	Failure behaviour
1	TLS version & cipher	3	0.20	None	N/A — local check
2	SNI analysis	4	0.15	None	N/A — local check
3	TCP & connection behaviour	5	0.15	Redis	Score=0 on Redis error
4	ASN classification	6	0.15	MaxMind DB (mmap)	Score=0 if DB unavailable
5	FCrDNS enrichment	7	0.10	DNS resolver	Score=0 on DNS error
6	Beaconing detector	9	0.10	Redis	Score=0 on Redis error
7	AbuseIPDB score	10	0.10	AbuseIPDB API	Score=0 on API error
8	RDAP org reputation	11	0.05	RDAP/WHOIS APIs	Score=0 on API error

Signal 1 — TLS Version and Cipher Check

Phase: 3 **Module:** `src/security/tls_enforcer.py`

What It Measures

This signal checks the TLS protocol version and cipher suite list from the ClientHello against a set of hard rules. TLS 1.0 and 1.1 are deprecated and have known vulnerabilities; SSLv3 is critically broken. Weak cipher suites indicate either an outdated client or deliberate downgrade probing.

Score Computation

When `security_policy.tls_version_bypass.enabled` is `true`, TLS 1.0/1.1/SSLv3 trigger an immediate TCP RST (before scoring). When the bypass is disabled, they contribute a RiskSignal instead:

Table 21: TLS version score contributions

Condition	Score	Notes
TLS 1.3, strong ciphers	0	Clean
TLS 1.2, ECDHE + AES-GCM / ChaCha20	0	Acceptable
TLS 1.2 with weak ciphers (RC4, DES, 3DES, NULL, EXPORT)	40	Routed through scorer
TLS 1.1 (bypass disabled)	70	Deprecated protocol
TLS 1.0 (bypass disabled)	80	Deprecated protocol
SSLv3 (bypass disabled)	95	Critically deprecated

Configuration Keys

`security_policy.tls_version_bypass.enabled` — when true, old TLS versions trigger TCP RST directly. When false, they contribute a risk score.

Failure Behaviour

This signal performs only local parsing of the ClientHello. There is no external service dependency. It cannot fail.

Signal 2 — SNI Analysis

Phase: 4 **Module:** `src/security/sni_analyzer.py`

What It Measures

Server Name Indication (SNI) is an extension in the ClientHello that specifies which hostname the client is trying to reach. Legitimate browsers always include SNI when connecting to HTTPS services. Missing SNI, IP-literal SNI, and algorithmically generated domain names are all strong signals of automated tools or malware.

Score Computation

DGA Detection Algorithm

Domain Generation Algorithm detection uses Shannon entropy of the domain label characters. A real domain like `example.com` has low

Table 22: SNI signal score contributions

Condition	Score	Notes
SNI present, matches expected hostname	0	Normal browser or API client
SNI present, unexpected hostname	10–30	May be misconfigured client or proxy
SNI absent	50	Scanners and tools typically omit SNI
SNI is an IP literal (e.g., `192.168.1.1`)	60	Invalid SNI; tools and scanners
SNI is DGA-like (high entropy, no real TLD)	40–80	Score based on entropy measurement

entropy; a DGA domain like `xk9p2mq4vrl.cn` has high entropy. The threshold is tunable:

- Entropy < 3.5 bits/character: clean
- Entropy 3.5–4.2 bits/character: moderate score (40)
- Entropy > 4.2 bits/character: high score (80)

Additionally, the TLD is checked against a list of TLDs commonly used in DGA campaigns. A domain matching a flagged TLD with high entropy scores at the upper end of the range.

Failure Behaviour

This signal performs only local computation. There is no external service dependency. It cannot fail.

Signal 3 — TCP and Connection Behaviour

Phase: 5 **Modules:** `src/security/tcp_analyzer.py`, `src/security/mtls.py`

What It Measures

TCP and connection behaviour signals capture how a client connects over time, not just what it sends. Automated tools have different connection patterns from real users: they connect more frequently, resume sessions less often, and maintain more simultaneous connections.

JA4T Fingerprinting

JA4T is the TLS transport fingerprint — a hash of the TCP window size, TCP options (MSS, window scale, SACK, timestamps), and their ordering. Unlike JA4 (which fingerprints the TLS application layer), JA4T fingerprints the TCP/IP stack implementation. Together, JA4 + JA4T can distinguish a real Chrome browser from a tool that has copied Chrome's JA4 value but runs on a different OS or with a modified TCP stack.

Score Components

Redis Keys Used

- `sess:{ip}` — Hash of `total` and `resumed` session counts
- `conn:{ip}` — Current concurrent connection count (INCR/DECR)

Table 23: TCP behaviour signal score components

Component	Score contribution	Notes
JA4T mismatch with claimed browser	+20	JA4T profile inconsistent with JA4 fingerprint label
Session resumption ratio $< 10\%$	+15	Legitimate browsers resume sessions aggressively
Concurrent connections > 20	+30	Legitimate browsers maintain 6–8 connections
Very short connection lifespan (< 0.1s)	+20	Scanner pattern — connect, sniff, disconnect
First-seen IP (no return visit history)	+5	Weak signal; most IPs start here
Return visitor with all blocked history	+25	IP that only ever got blocked

- `visit:{ip}` — Hash of `first_seen`, `total`, `allowed`, `blocked`

Failure Behaviour

If Redis is unreachable, all Redis-backed sub-signals return score=0. The signal as a whole returns score=0. The proxy logs the Redis error and increments `ja4proxy_redis_operations_total` with `result="error"`.

Signal 4 — ASN Classification

Phase: 6 **Module:** `src/security/asn_classifier.py`

What It Measures

The Autonomous System Number (ASN) associated with the client IP identifies the network organisation that owns the IP block. Legitimate browser traffic overwhelmingly originates from residential ISPs, mobile carriers, and corporate networks. Automated scanners and bots are disproportionately hosted in datacenters and cloud providers.

Score Computation

ASN classification uses:

1. The MaxMind GeoLite2 ASN database (memory-mapped for zero-copy lookup)
2. A curated datacenter ASN list in `config/asn_datacenter_list.yml`
3. The Tor exit node list (fetched periodically from `check.torproject.org`)

Failure Behaviour

If the MaxMind database file is missing or corrupt, this signal returns score=0. The proxy logs the failure (there is no dedicated error metric for a missing GeoIP database; `ja4proxy_signal_latency_seconds` records only timing). The GeoLite2 database should be updated monthly and validated at startup.

Table 24: ASN signal score contributions

ASN category	Score	Notes
Residential ISP	0	Expected for browser traffic
Mobile carrier	0	Expected for mobile browser traffic
Corporate network	5	Minor flag — APIs often originate here
Hosting / datacenter (general)	25	Common for cloud-hosted scanners
Known VPN provider	30	Often used to evade geo-blocking
Tor exit node	65	Significant signal — anonymisation tool
Known botnet/abuse ASN	80	From curated bad-org list

Signal 5 — FCrDNS Enrichment

Phase: 7 **Module:** `src/security/dns_enrichment.py`

What It Measures

Forward-confirmed reverse DNS (FCrDNS) validates that the PTR record for a client IP resolves to a hostname, and that hostname's A/AAAA record resolves back to the same IP. Residential ISPs, cloud providers, and CDNs all maintain consistent FCrDNS. Ephemeral scanner infrastructure typically does not.

Classification Logic

Table 25: FCrDNS signal classification and scores

Classification	Score	Notes
FCrDNS valid, residential pattern	0	PTR resolves to ISP hostname pattern
FCrDNS valid, cloud/datacenter	10	e.g. `ec2-*.compute.amazonaws.com`
PTR absent or NXDOMAIN	30	No reverse DNS — common for scanners
PTR present but FCrDNS fails	35	Inconsistent DNS — suspicious
PTR matches known scanning hostname	60	e.g. Shodan, Censys, known scanner infra

Async Lookup Architecture

DNS lookups are fire-and-forget: `asyncio.create_task(dns_lookup(ip))` is called at pipeline start. Results are written to a short-lived in-process cache. If the lookup has not completed when the scorer runs, the signal returns score=0 for this connection (fail open). Results are available for subsequent connections from the same IP.

Failure Behaviour

DNS resolution errors (NXDOMAIN, SERVFAIL, timeout) cause this signal to return score=0. The failure is logged at DEBUG level (DNS failures are common and non-actionable).

Signal 6 — Beaconing Detector

Phase: 9 **Module:** `src/security/beaconing_detector.py`

What It Measures

Beaconing is the pattern of regular, low-jitter connections from the same source — characteristic of C2 implants and automated scheduled tasks rather than human browsing. Human browsing has highly variable inter-arrival times (IAT). Beaconing tools produce IAT distributions with low coefficient of variation.

IAT Algorithm

1. Connection timestamps are stored in Redis Sorted Set `beacon:{ip}:{ja4}` (score = Unix timestamp in milliseconds)
2. The sliding window retains the last N timestamps within a 30-minute window
3. **Inter-arrival times** are computed as consecutive differences: $\text{IAT}_i = t_{i+1} - t_i$
4. The **coefficient of variation** (CV) is computed: $\text{CV} = \sigma/\mu$
5. Low CV indicates regular beaconing; high CV indicates human browsing

Table 26: Beaconing signal score thresholds

CV range	Score	Notes
< 5 connections in window	0	Insufficient data; no score assigned
CV > 0.8	0	High variability — human pattern
CV 0.4–0.8	20	Moderate regularity
CV 0.1–0.4	50	Strong beaconing pattern
CV < 0.1	75	Near-perfect regularity — almost certainly automated

Redis Keys Used

`beacon:{ip}:{ja4}` — Sorted Set where member = timestamp and score = timestamp. ZRANGEBYSCORE is used to retrieve the sliding window. ZADD appends new timestamps. Old members (outside the window) are removed with ZREMRANGEBYSCORE.

Failure Behaviour

If Redis is unreachable, this signal returns score=0. Redis writes are fire-and-forget and do not block the pipeline.

Signal 7 — AbuseIPDB Score

Phase: 10 **Module:** `src/security/abuseipdb.py`

What It Measures

AbuseIPDB is a community-maintained database of IP addresses reported for malicious activity (scanning, brute-force, spam, C2 traffic). Each IP has a confidence score (0–100) reflecting how frequently it has been reported and how recently.

Score Computation

Table 27: AbuseIPDB signal score mapping

AbuseIPDB confidence	Risk signal score	Notes
0–49	0	Below minimum threshold; no signal
50–74	30	Moderate abuse history
75–89	60	Significant abuse history
90–100	80	Very high confidence abuse
API error / timeout	0	Fail open — never increases score

Caching Architecture

AbuseIPDB responses are cached in Redis as `abuseipdb:{ip}` (JSON, TTL=3600s by default). Cache hits return immediately with zero API call. Cache misses trigger an async API call (fire-and-forget) and return score=0 for the current connection. The result is available for subsequent connections from the same IP.

This design means the *first* connection from a never-seen-before IP pays no AbuseIPDB penalty — it is allowed while the lookup runs. This is intentional (fail open).

Failure Behaviour

API errors, HTTP 429 rate limit responses, and network timeouts all produce score=0. The error is logged with context and the `ja4proxy_abuseipdb_lookups_total{result="error"}` counter is incremented. The proxy continues functioning with zero AbuseIPDB signal.

Signal 8 — RDAP Org Reputation

Phase: 11 **Module:** `src/security/rdap_enrichment.py`

What It Measures

RDAP (Registration Data Access Protocol) and WHOIS provide the organisation that owns an IP block. JA4proxy maintains a curated list of known-bad organisations in `config/known_bad_orgs.yml`. Matching the RDAP organisation name against this list provides a signal about whether the client IP is associated with organisations that systematically operate malicious infrastructure.

Score Computation

Table 28: RDAP signal score contributions

Condition	Score	Notes
Known-good org (major cloud, CDN)	0	AWS, GCP, Azure, Cloudflare, Akamai
Unknown org	5	Default for unrecognised registrants
Org name matches known-bad list	60–80	Score from `known_bad_orgs.yml`
RDAP lookup failed	0	Fail open

Block Expansion

When `rdap.block_expansion: true` (disabled by default), blocking an IP from a known-bad org triggers an automatic CIDR block of the entire /24 (IPv4) or /48 (IPv6) prefix containing that IP. The expanded block is written to Redis and picked up by the Layer 0d CIDR trie refresh.

Block Expansion Risk

Block expansion must never be enabled without manual review of the affected prefix. RDAP data is sometimes stale, incorrect, or shared between legitimate and malicious operators. A rogue /24 expansion affecting a major ISP block can lock out thousands of legitimate users.

WHOIS Pivot

The RDAP enrichment module also performs WHOIS pivoting: it looks up the registrant email domain for the IP block's registrant record. A registrant email from a known throwaway or bulletproof hosting domain adds an additional score contribution.

Failure Behaviour

RDAP API errors, timeouts, and malformed responses all return score=0. RDAP queries are cached in Redis (`rdap:{ip_or_prefix}`, TTL=86400s) to avoid repeated lookups for the same prefix.

Phase 203 Signals (Go Production Runtime)

Phase 203 closes five production-critical signal gaps in the Go proxy. These signals run *in addition* to the eight described above; they are documented here as a list because the full algorithmic detail is maintained in the phase document (`docs/phases/PHASE_203.md`) rather than reproduced in this reference manual. Each signal is independently configurable and follows the same fail-open contract as Signals 1–8.

TAP OS-mismatch (203a) Reads JA4T fingerprints written to Redis by the Phase 20 TAP node (key `fp:os:ip:{ip}`). When the JA4-claimed OS-class disagrees with the TAP-observed OS-class, a signal is emitted. The inline Go proxy cannot compute JA4T itself — by the time `accept()` returns, the kernel has consumed the SYN. If the TAP node is not deployed, the lookup returns nil and the signal is silently dormant (fail open). See `docs/phases/PHASE_203.md` §203a for the architectural rationale.

JA4 TLS-version mismatch (203b) Catches TLS-version spoofing where the JA4 prefix (`t13`/`t12`/...) disagrees with the TLS version actually negotiated. A common evasion pattern for tools that hand-craft a ClientHello matching a modern browser but cannot complete a real TLS 1.3 handshake.

Weak-cipher parity (203c) The Go `weakCipherSet` is now synchronised to *exactly* the 40 cipher suites enumerated in Python's `WEAK_CIPHERS`. A drift between the two implementations had previously let some weak suites score zero in Go that scored 40 in Python. Verified by a parity test against the Python oracle.

DGA parity (203d) The Go `dgaConfidence()` algorithm has been re-ported to match Python's `dga_score()` rule-for-rule. The Shannon-entropy thresholds, TLD list, and cap-out scoring now match across implementations.

Deep health (203e) `/health/deep` adds component checks for tarpit saturation and (optionally) GeoIP database freshness, with anti-flap hysteresis so transient blips do not flap the endpoint. `/health` (the k8s liveness probe) is deliberately left untouched and remains a tight TCP-level reachability check.

Phase 203 signals are Go-only

These five signals exist in the Go production runtime. The Python prototype contains the original implementations from which 203c (weak ciphers) and 203d (DGA) were ported; 203a/203b/203e are net-new in the Go runtime and have no Python counterpart.

Metrics Reference

All JA4proxy metrics are prefixed `ja4proxy_`. They are exposed on the `/metrics` HTTP endpoint (default port 9090) in Prometheus text format and scraped by the Prometheus instance at `:9090`.

Metric naming follows the Prometheus naming convention: `ja4proxy_{subsystem}_{metric_name}_{unit}`. Counters always end in `_total`. Histograms emit `_bucket`, `_count`, and `_sum` automatically. All metrics are registered at startup; a metric that has never been incremented will still appear in `/metrics` with a zero value.

Connection Metrics

ja4proxy_connections_total

Field	Value
Type	Counter
Labels	`action`: `allow` | `block` | `tarpit` | `ban` | `flag` | `rate_limit`
Description	Total connections processed, labelled by the final action taken

Table 29: `ja4proxy_connections_total` — connection outcome counter

Example PromQL:

```
# Current connection rate (all outcomes)
rate(ja4proxy_connections_total[5m])

# Block ratio
rate(ja4proxy_connections_total{action="block"}[5m])
  / rate(ja4proxy_connections_total[5m])

# Breakdown by action
sum by (action) (rate(ja4proxy_connections_total[1m]))
```

ja4proxy_connections_active

Type	Gauge
Labels	None
Description	Number of connections currently in the pipeline or forwarding phase

Table 30: `ja4proxy_connections_active` — current active connections gauge

Bypass Metrics

ja4proxy_bypass_total

Table 31: `ja4proxy_bypass_total` — fast-path bypass counter by bypass type

Type	Counter
Labels	`type`: `alpn` | `ja4_whitelist` | `mtls` | `static_ip` | `ja4_blacklist` | `country` | `spamhaus` | `tls_version`
Description	Total connections short-circuited by each bypass condition

Monitor Bypass Volumes

Monitor `ja4proxy_bypass_total` when modifying bypass configuration. A sudden drop in `alpn` bypass traffic may indicate HAProxy misconfiguration or an upstream change in ALPN negotiation.

Example PromQL:

```
# ALPN bypass rate
rate(ja4proxy_bypass_total{type="alpn"}[5m])

# Spamhaus blocks per minute
rate(ja4proxy_bypass_total{type="spamhaus"}[1m]) * 60
```

Risk Scoring Metrics

ja4proxy_risk_score

Table 32: `ja4proxy_risk_score` — score distribution histogram

Type	Histogram
Labels	None
Buckets	0, 10, 20, 30, 40, 50, 60, 70, 80, 90, 100
Description	Distribution of computed risk scores across all fully scored connections (bypassed connections are exclu

This histogram is the primary tool for evaluating threshold configuration before raising the dial. Plot the distribution in Grafana and select a threshold that captures known-bad traffic with minimal false positive exposure.

Example PromQL:

```
# 95th percentile score
histogram_quantile(0.95, rate(ja4proxy_risk_score_bucket[5m]))

# Fraction of connections scoring above 60
(
  sum(rate(ja4proxy_risk_score_bucket{le="100"}[5m]))
  - sum(rate(ja4proxy_risk_score_bucket{le="60"}[5m]))
)
/ sum(rate(ja4proxy_risk_score_bucket{le="100"}[5m]))
```

Table 33: `ja4proxy_dial_setting` — current dial value gauge

Type	Gauge
Labels	None
Description	Current value of the dial setting (0–100). Updated on every config reload.

ja4proxy_dial_setting

Signal Metrics

ja4proxy_signal_latency_seconds

Table 34: `ja4proxy_signal_latency_seconds` — per-signal latency histogram

<table>
<tr><td>Type</td><td>Histogram</td></tr>
<tr><td>Labels</td><td><code>signal</code>: <code>tls</code> | <code>sni</code> | <code>tcp</code> | <code>asn</code> | <code>dns</code> | <code>beacon</code> | <code>abuseipdb</code> | <code>rdap</code></td></tr>
<tr><td>Description</td><td>Time taken (in seconds) by each signal check, including cache hits</td></tr>
</table>

Example PromQL:

```
# 99th percentile AbuseIPDB lookup latency
histogram_quantile(0.99,
  rate(ja4proxy_signal_latency_seconds_bucket{signal="abuseipdb"}[5m])
)

# Mean signal latency by type
rate(ja4proxy_signal_latency_seconds_sum[5m])
/ rate(ja4proxy_signal_latency_seconds_count[5m])
```

ja4proxy_signal_score

Table 35: `ja4proxy_signal_score` — last-seen score per signal

Type	Gauge
Labels	`signal`: same set as above
Description	Most recent score contributed by each signal. Useful for debugging individual signal behaviour. Update

External Service Metrics

ja4proxy_abuseipdb_lookups_total

Example PromQL:

```
# AbuseIPDB cache hit rate
rate(ja4proxy_abuseipdb_lookups_total{result="hit"}[5m])
/ rate(ja4proxy_abuseipdb_lookups_total[5m])

# AbuseIPDB error rate alert (should be near zero)
rate(ja4proxy_abuseipdb_lookups_total{result="error"}[5m]) > 0.1
```

Table 36: `ja4proxy_abuseipdb_lookups_total`

Type	Counter
Labels	`result`: `hit` (cache hit) | `miss` (cache miss → API call) | `error` (API error)
Description	Total AbuseIPDB cache operations

Table 37: `ja4proxy_rdap_lookups_total`

Type	Counter
Labels	`result`: `hit` | `miss` | `error`
Description	Total RDAP cache operations

ja4proxy_rdap_lookups_total

Infrastructure Metrics

ja4proxy_redis_operations_total

ja4proxy_config_reloads_total

Fingerprint Metrics

ja4proxy_fingerprint_seen_total

Ban Metrics

ja4proxy_ban_ttl_seconds

Alert Rule Examples

The following Prometheus alert rules provide a starting point for production alerting:

```
groups:
  - name: ja4proxy
    rules:
      # Redis connectivity – any failure is critical
      - alert: JA4proxyRedisErrors
        expr: rate(ja4proxy_redis_operations_total{result="error"}[2m]) > 0.5
        for: 1m
        labels:
          severity: critical
        annotations:
          summary: "JA4proxy Redis errors ({{ $value }}/s)"
          description: >
            JA4proxy is experiencing Redis errors. The fail-open
            guarantee means connections are still being allowed,
            but bans, rate limits, and signals are not being applied.

      # Blocking rate spike – may indicate attack or misconfiguration
      - alert: JA4proxyHighBlockRate
        expr: |
          rate(ja4proxy_connections_total{action="block"}[5m])
            / rate(ja4proxy_connections_total[5m]) > 0.3
```

Table 38: `ja4proxy_redis_operations_total` — Redis operations by command and result

Type	Counter
Labels	`command`: the Redis command (e.g. `GET`, `ZADD`, `XADD`); `result`: `ok` or `error`
Description	Total Redis operations by command and result. Filter `result="error"` for failures; a sustained non-zero e

Table 39: `ja4proxy_config_reloads_total`

Type	Counter
Labels	`result`: `success` | `error`
Description	Total config reload attempts. An `error` result means the new config was invalid and the proxy continues

```
        for: 5m
        labels:
          severity: warning
        annotations:
          summary: "JA4proxy block rate > 30%"
          description: >
            More than 30% of connections are being blocked.
            Verify this is not a false positive wave.

      # AbuseIPDB API errors - degraded enrichment
      - alert: JA4proxyAbuseIPDBErrors
        expr: rate(ja4proxy_abuseipdb_lookups_total{result="
    error"}[5m]) > 0.1
        for: 5m
        labels:
          severity: warning
        annotations:
          summary: "AbuseIPDB lookup errors"

      # No connections - possible proxy failure
      - alert: JA4proxyNoConnections
        expr: rate(ja4proxy_connections_total[5m]) == 0
        for: 2m
        labels:
          severity: critical
        annotations:
          summary: "JA4proxy receiving no connections"
```

Grafana Dashboard Panels

The following panels are recommended for the JA4proxy operational dashboard:

Table 40: `ja4proxy_fingerprint_seen_total` — fingerprint observation counter

Type	Counter
Labels	`name`: human-readable fingerprint label from `security.fingerprint_labels`, e.g. `Chrome 120+`, `Tool/Bot (TLS`
Description	Total connections seen by named fingerprint category. Useful for tracking browser vs tool traffic ratios.

Table 41: `ja4proxy_ban_ttl_seconds` — ban TTL distribution histogram

Type	Histogram
Labels	None
Description	Distribution of ban TTLs at the time the ban is issued. Escalating bans produce higher TTL values over

Table 42: Recommended Grafana dashboard panels

Panel name	**Visualisation**	**PromQL query**
Connection rate by action	Stacked time series	`sum by (action) (rate(ja4proxy_connections_total[1m]))`
Risk score heatmap	Heatmap	`rate(ja4proxy_risk_score_bucket[1m])`
Block rate %	Gauge (0–100%)	Block / total connections
Bypass breakdown	Pie chart	`sum by (type) (rate(ja4proxy_bypass_total[5m]))`
Signal latency P99	Bar chart	P99 histogram for each signal
Fingerprint traffic mix	Pie chart	`sum by (name) (rate(ja4proxy_fingerprint_seen_total[5m]))`
Dial setting	Stat panel	`ja4proxy_dial_setting`
Redis error rate	Time series	`rate(ja4proxy_redis_operations_total{result="error"}[1m])`
AbuseIPDB cache hit rate	Stat panel	hit / total rate

Redis Schema

Redis is the shared state layer. Every JA4proxy instance reads and writes the same Redis instance. All bans, rate windows, beaconing timestamps, JA4 lists, and enrichment caches live here. The local in-process cache is a performance overlay — Redis is the source of truth.

REDIS DATA STRUCTURE SELECTION is not arbitrary. Each use case has a specific rationale — wrong data structures produce incorrect sliding windows, lost updates, or excessive memory usage. This chapter documents every key pattern, its data structure, and the rationale for that choice.

Data Structure Rationale

Never Use Redis for CIDR Matching

CIDR matching in Redis requires iterating all keys matching a pattern or maintaining a sorted set of prefixes. Both approaches are O(n) for large blocklists. The in-process trie (pytricia for Python, prefix trie for Go) provides O(log n) lookup and is the only correct approach. All CIDR matching is done in-process; Redis stores the list for sharing across instances and reloading after restart.

JA4 Fingerprint Lists

ja4:blacklist

At startup, the config loader reads `security.blacklist` from `proxy.yml` and executes a Redis pipeline to `SADD` all entries to `ja4:blacklist`. On hot-reload, entries removed from config are `SREM`'d. An in-process set mirrors the Redis SET for zero-latency lookup on the hot path.

ja4:whitelist

IP Bans

ban:{ip}

Ban TTL follows an escalation schedule: initial TTL is `ban_duration`

Table 43: Redis data structure selection rationale

Use case	Structure	Why this structure	Alternative co
JA4 blacklist/whitelist	SET	O(1) SISMEMBER; small and static	Hash — unne head
IP bans	String + TTL	Per-key TTL required; atomic SETEX	SET — no per-l
Sliding window rate limiting	Sorted Set + Lua (EVALSHA)	True sliding window; no boundary errors	INCR + EXPIRI dow only
Beaconing timestamps	Sorted Set (score=timestamp)	ZRANGEBYSCORE for time windows; ordered	List — no time
Session resumption ratio	Hash (total, resumed)	Atomic HINCRBY; two fields one key	Two strings – trips
Concurrent connection count	String (INCR/DECR)	Simple atomic counter	Hash — unnec
Return visitor tracking	Hash (first_seen, total, allowed, blocked)	Multi-field HINCRBY; atomic	Multiple keys –
Unique IP per CIDR	HyperLogLog (PFADD/PFCOUNT)	O(1), 12KB/key, 0.81% error	Hash of IPs – memory
Enrichment dedup	Bloom filter (BF.ADD/BF.EXISTS)	False positives acceptable; O(1)	SET — exact b memory
CIDR matching	In-process trie (NOT Redis)	No CIDR primitive in Redis; O(log n) trie	Redis SCAN –
GeoIP/ASN lookup	In-process mmap (NOT Redis)	MaxMind designed for mmap	Redis hash — optimisation
Cross-instance events	Stream (XADD/XREADGROUP)	Persistent, replayable; consumer groups	Pub/Sub — epl play
Analytics findings	String + TTL	Read by proxy; simple JSON blob	Stream — adds

Table 44: `ja4:blacklist` key reference

Key	`ja4:blacklist`
Type	SET
TTL	None (persistent)
Writer	Config loader at startup; Management UI
Reader	Pipeline layer 2 (JA4 blacklist bypass)
Example members	`t13d190900_9dc949149365_97f8aa674fd9`
Operations	`SADD`, `SREM`, `SISMEMBER`, `SMEMBERS`

(default 300s); subsequent bans double the TTL up to a maximum of 604800s (7 days). The current escalation level is stored in the JSON value's `escalation` field.

Rate Limiting Windows

rate:{strategy}:{key}

The sliding window is implemented as a Lua script that atomically:

1. Removes entries older than the window size (ZREMRANGEBYSCORE)
2. Adds the current timestamp (ZADD)
3. Returns the current window count (ZCOUNT)

Using Lua (EVALSHA) ensures the three operations are atomic

Table 45: `ja4:whitelist` key reference

Key	`ja4:whitelist`
Type	SET
TTL	None (persistent)
Writer	Config loader at startup; Management UI
Reader	Pipeline layer 1b (JA4 whitelist bypass)
Example members	`t13d1516h2_8daaf6152771_02713d6af862`

Table 46: `ban:{ip}` key reference

Key pattern	`ban:{ip}` where `{ip}` is the canonical IP string
Type	String
Value	JSON: `{"banned_at": 1712232896, "reason": "rate_limit_by_ip", "ttl": 300}`
TTL	`ban_duration` seconds (escalates on re-ban)
Writer	Action decider (ban action)
Reader	Pipeline layer 0 (early ban check); local LRU cache
Examples	`ban:185.220.101.1`, `ban:2a06:98c1:3120::3`
Operations	`SET key value EX ttl`, `GET`, `DEL`

from Redis's perspective — no race condition is possible between the remove and count steps.

Beaconing Data

beacon:{ip}:{ja4}

Connection State

conn:{ip}

visit:{ip}

sess:{ip}

HyperLogLog CIDR Density

hll:cidr24:{cidr} and hll:cidr48:{cidr}

HyperLogLog is the only correct data structure for this use case. Tracking all distinct IPs in a /24 in a Hash would require up to 256 entries per key; in a /48 it would be unbounded. HyperLogLog provides an accurate-enough count with fixed memory cost.

Enrichment Caches

abuseipdb:{ip}

rdap:{ip_or_prefix}

geo:{ip}

Analytics Infrastructure

events — The Event Stream

Every connection that completes the scoring pipeline produces one

Table 47: `rate:{strategy}:{key}` key reference

Key pattern	`rate:{strategy}:{key}`
Type	Sorted Set
Score	Unix timestamp in milliseconds
Member	Connection UUID (ensures uniqueness)
TTL	Managed by ZREMRANGEBYSCORE; no explicit key TTL
Writer	Pipeline rate-limiting layer
Reader	Pipeline rate-limiting layer (ZCOUNT)
Example keys	`rate:by_ip:185.220.101.1`
	`rate:by_ja4:t13d190900_...`
	`rate:by_ip_ja4_pair:185.220.101.1:t13d190900_...`
Operations	Lua EVALSHA for atomic ZADD + ZREMRANGEBYSCORE + ZCOUNT

Table 48: `beacon:{ip}:{ja4}` key reference

Key pattern	`beacon:{ip}:{ja4}`
Type	Sorted Set
Score	Unix timestamp in milliseconds
Member	Unix timestamp (same as score — member must be unique)
TTL	Managed by ZREMRANGEBYSCORE (30-minute sliding window)
Writer	Beaconing detector signal
Reader	Beaconing detector signal (ZRANGEBYSCORE)
Example key	`beacon:185.220.101.1:t13d1900...`

stream entry. Bypassed connections (ALPN, whitelist) also produce a minimal entry for volume tracking. The analytics node processes entries in batch using consumer groups, allowing multiple analytics workers to process in parallel without duplicate processing.

Why Streams not Pub/Sub

Redis Pub/Sub is ephemeral — a message published while a subscriber is down is lost. Redis Streams persist messages and allow consumer groups with acknowledgements. After a restart or deployment, the analytics node replays missed events from where it left off. This is critical for campaign detection, which requires historical context.

findings:{ip}

Management and Audit

management:policy_audit

Every change to `security_policy` is written here. Changes via the Management UI are attributed to the operator's session IP. Changes via `SIGHUP` config reload are attributed to `"config_reload"`. This list provides the complete audit trail for compliance purposes.

Key Namespace Summary

Table 49: `conn:{ip}` key reference — concurrent connections

Key pattern	`conn:{ip}`
Type	String (integer)
TTL	Managed by INCR/DECR lifecycle; expires after last connection closes
Writer	TCP connection handler (INCR on accept, DECR on close)
Reader	TCP behaviour signal
Example	`conn:185.220.101.1` → `"42"`

Table 50: `visit:{ip}` key reference — return visitor tracking

Key pattern	`visit:{ip}`
Type	Hash
Fields	`first_seen` (Unix timestamp), `total` (integer), `allowed` (integer), `blocked` (integer)
TTL	None (persistent for now; rotation policy TBD)
Writer	Action decider (HINCRBY on each action)
Reader	TCP behaviour signal

Table 51: `sess:{ip}` key reference — session resumption ratio

Key pattern	`sess:{ip}`
Type	Hash
Fields	`total` (integer), `resumed` (integer)
TTL	None (persistent)
Writer	TLS handshake analyser
Reader	TCP behaviour signal (ratio = resumed/total)

Table 52: HyperLogLog CIDR density keys

Key patterns	`hll:cidr24:{cidr}` (IPv4 /24), `hll:cidr48:{cidr}` (IPv6 /48)
Type	HyperLogLog
Operations	`PFADD` (add IP), `PFCOUNT` (count unique IPs)
Memory	Fixed 12KB per key, regardless of distinct IP count
Error rate	0.81% standard error
Example key	`hll:cidr24:185.220.101.0/24`
Writer	IP normalisation layer
Reader	Analytics node (CIDR density signals)
TTL	30 days

Table 53: `abuseipdb:{ip}` key reference

Key pattern	`abuseipdb:{ip}`
Type	String
Value	JSON: `{"score": 85, "cached_at": 1712232896, "categories": [14, 21]}`
TTL	`abuseipdb.cache_ttl` seconds (default 3600)
Writer	AbuseIPDB signal (async, after API response)
Reader	AbuseIPDB signal (cache check)

Table 54: `rdap:{ip_or_prefix}` key reference

Key pattern	`rdap:{ip_or_prefix}`
Type	String
Value	JSON: `{"org": "Example ISP", "prefix": "185.220.0.0/16", "country": "DE"}`
TTL	86400 seconds (24h)
Writer	RDAP enrichment signal
Reader	RDAP enrichment signal (cache check)
Example key	`rdap:185.220.101.1`, `rdap:185.220.0.0/16`

Table 55: `geo:{ip}` key reference — GeoIP country cache

Key pattern	`geo:{ip}`
Type	String
Value	ISO 3166-1 alpha-2 country code, e.g. `"RU"`
TTL	86400 seconds (24h)
Writer	GeoIP lookup layer
Reader	GeoIP country block layer

Table 56: `events` Redis Stream key reference

Key	`events`
Type	Stream
Writer	Proxy (XADD, fire-and-forget)
Reader	Analytics node (XREADGROUP with consumer group `analytics`)
Retention	MAXLEN 100000 entries (approximate, using `XADD MAXLEN ~`)
Operations	`XADD events * field1 val1 ...` (write), `XREADGROUP GROUP analytics consumer1 COUNT 100 STREAMS events >`

Table 57: `findings:{ip}` key reference — analytics findings feed-back

Key pattern	`findings:{ip}`
Type	String
Value	JSON: `{"campaign": "scan-wave-14", "attribution_score": 80, "coordinated": true}`
TTL	300 seconds
Writer	Analytics node
Reader	Pipeline layers 11 and 12 (attacker attribution, behavioural analysis)

Table 58: `management:policy_audit` key reference

Key	`management:policy_audit`
Type	LIST
Value	JSON entries: `{"timestamp": "...", "changed_by": "admin@10.0.4.2", "item": "dial", "old_value": 0`
TTL	None (persistent; max 1000 entries via LTRIM)
Writer	Management UI (config changes); config reload process
Reader	Management UI audit log view
Operations	`LPUSH` (prepend new entry), `LTRIM 0 999` (keep last 1000)

Table 59: Complete Redis key namespace

Prefix	**Type**	**TTL**	**Purpose**
`ja4:blacklist`	SET	None	JA4 fingerprint blacklist
`ja4:whitelist`	SET	None	JA4 fingerprint whitelist
`ban:{ip}`	String	300s–604800s	IP ban with escalating TTL
`rate:{strategy}:{key}`	Sorted Set	Managed	Sliding window rate limits
`beacon:{ip}:{ja4}`	Sorted Set	Managed	Beaconing IAT timestamps
`conn:{ip}`	String (int)	Managed	Concurrent connection count
`visit:{ip}`	Hash	None	Return visitor history
`sess:{ip}`	Hash	None	Session resumption ratio
`hll:cidr24:{cidr}`	HyperLogLog	30d	Unique IPs per /24 (IPv4)
`hll:cidr48:{cidr}`	HyperLogLog	30d	Unique IPs per /48 (IPv6)
`abuseipdb:{ip}`	String (JSON)	3600s	AbuseIPDB score cache
`rdap:{ip}`	String (JSON)	86400s	RDAP org lookup cache
`geo:{ip}`	String	86400s	GeoIP country cache
`events`	Stream	MAXLEN 100k	Proxy event stream
`findings:{ip}`	String (JSON)	300s	Analytics findings feed-back
`management:policy_audit`	List	None (max 1000)	Policy change audit log

Log Format Reference

JA4proxy emits structured JSON logs to stdout. Every connection produces exactly one connection log entry at the action decision point. Additional entries are emitted for config reloads, errors, and policy events.

JSON is the primary log format. The human-readable text format is a secondary view, derived from the JSON, suitable for interactive terminal use. All SIEM integrations, log aggregation (Loki), and alerting pipelines should use the JSON format.

JSON Log Schema

Complete Field Reference

The signals Object

When present, the `signals` field contains the individual score contribution from each signal:

```
"signals": {
  "tls":       0,
  "sni":       0,
  "tcp":       0,
  "asn":       15,
  "dns":       0,
  "beacon":    0,
  "abuseipdb": 0,
  "rdap":      0
}
```

Each value is the raw signal score (0–100) before confidence weighting. A score of 0 means either the signal found nothing suspicious, or the external service was unavailable (fail open — both produce 0).

Example Log Entries

Allowed Browser Connection (ALPN bypass)

```
{
  "timestamp": "2026-04-04T12:34:56.789Z",
  "event": "connection",
```

```
  "action": "allow",
  "client_ip": "172.19.0.10",
  "country": "IE",
  "ja4": "t13d1113h2_8daaf6152771_02713d6af862",
  "fingerprint_name": "Browser (TLS 1.3)",
  "tls_version": "TLS 1.3",
  "sni": "example.com",
  "alpn": "h2",
  "dial": 0,
  "bypass": "alpn_browser",
  "reason": "Browser ALPN bypass",
  "duration_ms": 0.8
}
```

Note: `risk_score` and `signals` are absent because the connection was bypassed before scoring.

Blocked Connection (Spamhaus DROP match)

```
{
  "timestamp": "2026-04-04T12:35:10.421Z",
  "event": "connection",
  "action": "block",
  "client_ip": "185.220.101.1",
  "country": "DE",
  "ja4": "t13d1900h2_8daaf6152771_97f8aa674fd9",
  "fingerprint_name": "Tool/Bot (TLS 1.3)",
  "tls_version": "TLS 1.3",
  "sni": null,
  "dial": 75,
  "bypass": "spamhaus",
  "reason": "Spamhaus DROP match: 185.220.96.0/21",
  "duration_ms": 0.3
}
```

Scored and Tarpitted Connection

```
{
  "timestamp": "2026-04-04T12:36:42.100Z",
  "event": "connection",
  "action": "tarpit",
  "client_ip": "45.142.212.100",
  "country": "RU",
  "ja4": "t13d190900_9dc949149365_97f8aa674fd9",
  "ja4x": "a5c0d0f3d1b2_0e8a7c6b5d4e",
  "ja4t": "8192_2_1460_nWS",
  "fingerprint_name": "Sliver C2",
  "tls_version": "TLS 1.3",
  "sni": "cdn-185.example.com",
  "risk_score": 87,
  "dial": 75,
  "reason": "Score 87 exceeds tarpit threshold",
  "duration_ms": 2.4,
```

```
  "signals": {
    "tls":       0,
    "sni":       40,
    "tcp":       25,
    "asn":       65,
    "dns":       35,
    "beacon":    50,
    "abuseipdb": 80,
    "rdap":      60
  }
}
```

Banned Connection (Subsequent Request)

```
{
  "timestamp": "2026-04-04T12:37:05.002Z",
  "event": "connection",
  "action": "block",
  "client_ip": "45.142.212.100",
  "country": "RU",
  "dial": 75,
  "bypass": "ban_cache",
  "reason": "Banned for 604800s (escalation level 3)",
  "duration_ms": 0.1
}
```

Subsequent connections from a banned IP are rejected at layer 0 from the local ban cache. No JA4 parsing occurs, hence the absent fingerprint fields.

Config Reload Event

```
{
  "timestamp": "2026-04-04T13:00:00.000Z",
  "event": "config_reload",
  "action": null,
  "result": "success",
  "changed_keys": ["dial", "security_policy.spamhaus_bypass.
    enabled"],
  "duration_ms": 45.2
}
```

Bypass Disabled Warning

```
{
  "timestamp": "2026-04-04T08:00:00.001Z",
  "event": "bypass_disabled",
  "bypass": "alpn_browser_bypass",
  "effect": "browser traffic will be scored; false positive
    risk elevated",
  "level": "WARN"
```

```
}
```

Human-Readable Text Format

The text format is a condensed single-line summary derived from the JSON event. It is written to a separate log stream (or can be toggled on/off) for operator convenience during interactive sessions.

```
ALLOWED:  172.19.0.10 | Country: IE  | JA4: t13d1113h2_... |
    Name: Browser (TLS 1.3) | TLS: TLS 1.3
BLOCKED:  185.220.101.1 | Country: DE | Reason: Spamhaus DROP:
    185.220.96.0/21
TARPIT:   45.142.212.100 | Country: RU | JA4: t13d190900_... |
    Name: Sliver C2 | Score: 87
BANNED:   45.142.212.100 | Country: RU | Reason: Banned for
    604800s (level 3)
```

Log Levels

Loki Log Queries (LogQL)

Assuming logs are shipped to Loki with the label `job="ja4proxy"`, the following LogQL queries cover common operational needs:

```
# All blocked connections in last hour
{job="ja4proxy"} | json | action="block"

# All connections from a specific IP
{job="ja4proxy"} | json | client_ip="185.220.101.1"

# Connections with risk_score > 80
{job="ja4proxy"} | json | risk_score > 80

# All Spamhaus bypasses
{job="ja4proxy"} | json | bypass="spamhaus"

# Tarpitted connections with their signal breakdown
{job="ja4proxy"} | json | action="tarpit"
  | line_format "{{.client_ip}} score={{.risk_score}}"

# Config reload events
{job="ja4proxy"} | json | event="config_reload"

# Error events only
{job="ja4proxy"} | json | level="ERROR"

# Fingerprint distribution for last 24h
{job="ja4proxy"} | json | fingerprint_name != ""
  | unwrap risk_score
  | rate by (fingerprint_name) [5m]
```

SIEM Integration — CEF Format

For SIEM platforms that require Common Event Format (CEF), the following mapping converts JA4proxy JSON fields to CEF:

Example CEF line for a tarpitted connection:

```
CEF:0|JA4proxy|JA4proxy|1.0|connection_tarpit|connection_tarpit
    |8|
  src=45.142.212.100 act=tarpit cs1Label=JA4
  cs1=t13d190900_9dc949149365_97f8aa674fd9
  cs2Label=Country cs2=RU cs3Label=Reason
  cs3=Score 87 exceeds tarpit threshold cn1Label=Score
  cn1=87 cn2Label=Dial cn2=75
  start=2026-04-04T12:36:42.100Z
```

Table 60: JSON log field reference — connection event

Field	Type	Always present	Description
`timestamp`	string (ISO 8601)	Yes	UTC timestamp with milliseconds: `2026-04-04T12:34:56.789Z`
`event`	string	Yes	Event type: `connection`, `config_reload`, `error`, `bypass_disabled`
`action`	string	Yes	Final action: `allow`, `block`, `tarpit`, `ban`, `flag`, `rate_limit`
`client_ip`	string	Yes	Canonical client IP (from PROXY protocol). Full IPv4 or IPv6 string.
`country`	string	No	ISO 3166-1 alpha-2 country code. Absent if GeoIP lookup failed.
`ja4`	string	No	JA4 fingerprint string. Absent if ClientHello could not be parsed.
`ja4x`	string	No	JA4X extended fingerprint. Absent if not computed.
`ja4t`	string	No	JA4T TCP fingerprint. Absent if not computed.
`fingerprint_name`	string	No	Human-readable label for JA4, from `security.fingerprint_labels`
`tls_version`	string	No	TLS version string: `TLS 1.3`, `TLS 1.2`, etc.
`sni`	string	No	Server Name Indication from ClientHello. Absent if not present in handshake.
`alpn`	string	No	First ALPN value: `h2`, `h1`, etc.
`risk_score`	integer	No	Computed risk score (0–100). Absent for bypassed connections.
`dial`	integer	Yes	Current dial setting at decision time.
`bypass`	string	No	Bypass type if connection was bypassed: `alpn_browser`, `ja4_whitelist`, etc.
`reason`	string	Yes	Human-readable reason for the action.
`duration_ms`	float	Yes	Pipeline execution time in milliseconds.
`signals`	object	No	Per-signal scores as a JSON object. Absent for bypassed connections.

Level	Volume	Emitted for
ERROR	Low	Redis connection failure, config parse failure, TLS parser crash, external API repeated failure
WARN	Low	Bypass disabled at startup, AbuseIPDB rate limit hit, RDAP block expansion triggered
INFO	Medium	Config reload (success or error), startup completion, shutdown
DEBUG	High	Individual signal scores, cache hits/misses, Redis operations. Disabled in production.
(connection)	Very high	Every connection event — not strictly a log level; always emitted

Table 61: Log levels and when each is emitted

CEF field	JA4proxy JSON field	Notes
`CEF:0|JA4proxy|JA4proxy|1.0`	—	CEF header (vendor | product | version)
`name`	`event` + `action`	e.g. `connection_block`
`severity`	`risk_score` / 10	Scale: 0–10
`src`	`client_ip`	Source IP
`cs1` (JA4)	`ja4`	Custom string field
`cs2` (Country)	`country`	Custom string field
`cs3` (Reason)	`reason`	Custom string field
`cn1` (Score)	`risk_score`	Custom number field
`cn2` (Dial)	`dial`	Custom number field
`act`	`action`	Device action
`start`	`timestamp`	Event time
`deviceProcessName`	`"ja4proxy"`	Fixed string

Table 62: CEF field mapping for JA4proxy connection events

Deployment Reference

JA4proxy is deployed as a set of Docker containers orchestrated by Docker Compose (development and single-host production) or Helm (Kubernetes multi-host production). This chapter documents every service, environment variable, port, and network requirement.

The reference deployment topology places JA4proxy between a HAProxy load balancer and the backend origin server. All traffic from the internet passes through HAProxy (:443), which forwards raw TCP streams (with PROXY protocol v2 headers) to JA4proxy instances (:8080). JA4proxy then proxies allowed connections to the backend (:443).

Docker Compose Reference

Services Overview

Table 63: Docker Compose services

Service	Image	Ports	Role
`haproxy`	`haproxy:2.9-alpine`	`0.0.0.0:443:443`	TLS passthrough LB; injects PROXY protocol v2
`proxy`	`ja4proxy:latest`	`127.0.0.1:8080:8080`	JA4proxy pipeline; exposes metrics on :9090
`redis`	`redis:7.2-alpine`	`127.0.0.1:6379:6379`	Shared state; NOT exposed to internet
`analytics`	`ja4analytics:latest`	—	Redis Streams consumer; findings writer
`prometheus`	`prom/prometheus:v2.51.0`	`127.0.0.1:9090:9090`	Metrics collection
`grafana`	`grafana/grafana:10.4.0`	`127.0.0.1:3001:3000`	Dashboards
`loki`	`grafana/loki:2.9.5`	`127.0.0.1:3100:3100`	Log aggregation
`alertmanager`	`prom/alertmanager:v0.27.0`	`127.0.0.1:9093:9093`	Alert routing
`backend`	(operator-provided)	—	Protected origin; only reachable from proxy

Port Binding Policy

Management services (Prometheus, Grafana, Loki, Alertmanager) bind to `127.0.0.1` only. They must never be exposed on `0.0.0.0` in production. Redis binds to `127.0.0.1`. Only HAProxy binds to `0.0.0.0:443`. The proxy service binds to `127.0.0.1:8080`

> — it receives connections only from HAProxy, which is on the same host or the same Docker network.

Docker Networks

Table 64: Docker Compose network definitions

Network	Subnet	Connected services	Purpose
`dmz`	172.20.0.0/24	haproxy, proxy	HAProxy → proxy path
`app`	172.21.0.0/24	proxy, redis, analytics, backend	proxy → redis, backend
`monitoring`	172.22.0.0/24	proxy, prometheus, loki	Metrics and log scraping
`mgmt`	172.23.0.0/24	prometheus, grafana, alertmanager, loki	Observability stack internal

Environment Variables

The following environment variables must be set in the deployment environment (e.g. in a `.env` file alongside `docker-compose.yml`).

Table 65: Required and optional environment variables

Variable	Required	Default	Description
`REDIS_PASSWORD`	Yes	—	Redis AUTH password. Set on both proxy and redis services.
`ABUSEIPDB_API_KEY`	No	—	AbuseIPDB API key. If unset, AbuseIPDB lookups are disabled.
`JA4_DIAL`	No	`0`	Override dial setting at container start (useful for testing).
`JA4_LOG_LEVEL`	No	`INFO`	Log verbosity: `DEBUG`, `INFO`, `WARN`, `ERROR`
`JA4_BACKEND_HOST`	No	`backend`	Backend hostname override. Useful when backend DNS differs.
`JA4_BACKEND_PORT`	No	`443`	Backend port override.
`GF_SECURITY_ADMIN_PASSWORD`	No	`admin`	Grafana admin password. Change before production deployment.
`GF_SERVER_ROOT_URL`	No	—	Grafana public URL for link generation.
`MAXMIND_LICENSE_KEY`	Conditional	—	MaxMind license key for GeoLite2 database download. Required if using auto-download.

Volume Mounts

HAProxy Configuration

HAProxy must be configured for TLS passthrough with PROXY protocol v2 injection. The critical sections of `haproxy.cfg`:

Table 66: Docker volume mounts

Service	Host path	Container path	Notes
proxy	./config	/app/config	Read-only (RO). Config files.
proxy	./certs	/app/certs	Read-only. mTLS CA and certs.
proxy	./data/geoip	/app/data/geoip	Read-only. MaxMind DB files.
redis	./data/redis	/data	Redis persistence (RDB/AOF).
prometheus	./data/prometheus	/prometheus	Prometheus TSDB.
grafana	./data/grafana	/var/lib/grafana	Grafana dashboards and state.
loki	./data/loki	/loki	Loki log chunks.

```
frontend tls_in
    bind *:443
    mode tcp
    option tcplog
    default_backend ja4proxy

backend ja4proxy
    mode tcp
    option tcp-check
    server proxy1 proxy:8080 check send-proxy-v2
    # send-proxy-v2 enables PROXY protocol v2 header injection
    # TLS is NOT terminated - raw TCP passthrough
```

TLS Passthrough — Not TLS Termination

HAProxy must be configured in TCP mode (`mode tcp`), not HTTP mode (`mode http`). HTTP mode would terminate the TLS connection and re-encrypt it, defeating the purpose of JA4proxy. In TCP mode, HAProxy passes the raw TLS bytes through to JA4proxy unchanged. Only the PROXY protocol header is added by HAProxy.

Helm Chart Reference

Chart Structure

The JA4proxy Helm chart is located at `helm/ja4proxy/`. It deploys:

- A `Deployment` for the proxy pods
- A `Service` (ClusterIP) exposing port 8080
- A `ConfigMap` containing `proxy.yml`
- A `Secret` for Redis password and API keys
- A `ServiceMonitor` for Prometheus Operator scraping
- (Optional) A `HorizontalPodAutoscaler`

values.yaml Reference

Kubernetes Resources Created

```
# Install
helm install ja4proxy helm/ja4proxy \
```

Table 67: Key Helm chart values

Key	Type	Default	Description
`replicaCount`	integer	`2`	Number of proxy pod replicas
`image.repository`	string	`ja4proxy`	Container image repository
`image.tag`	string	`latest`	Image tag. Pin to a specific version in production.
`proxy.backendHost`	string	*required*	Backend service hostname
`proxy.backendPort`	integer	`443`	Backend port
`proxy.dial`	integer	`0`	Initial dial setting
`redis.url`	string	*required*	Redis connection URL
`redis.existingSecret`	string	`""`	Kubernetes Secret name containing `redis-password` key
`abuseipdb.enabled`	boolean	`true`	Enable AbuseIPDB lookups
`abuseipdb.existingSecret`	string	`""`	Secret containing `api-key`
`resources.requests.cpu`	string	`100m`	CPU request per pod
`resources.requests.memory`	string	`128Mi`	Memory request per pod
`resources.limits.cpu`	string	`1000m`	CPU limit
`resources.limits.memory`	string	`256Mi`	Memory limit
`autoscaling.enabled`	boolean	`false`	Enable HPA
`autoscaling.minReplicas`	integer	`2`	Minimum replicas
`autoscaling.maxReplicas`	integer	`10`	Maximum replicas
`autoscaling.targetCPUUtilizationPercentage`	integer	`70`	HPA CPU target
`serviceMonitor.enabled`	boolean	`true`	Create ServiceMonitor for Prometheus Operator
`geoip.volumeClaimName`	string	`""`	PVC name for GeoIP database files (read-only mount)

```
  --set proxy.backendHost=backend-svc.app.svc.cluster.local \
  --set redis.url=redis://redis-svc.data.svc.cluster.local:6379 \
  --set redis.existingSecret=ja4proxy-secrets

# Upgrade (e.g., raise dial)
helm upgrade ja4proxy helm/ja4proxy --set proxy.dial=50
```

Networking Requirements

Firewall Rules for DMZ Deployment

Production Deployment Checklist

The following checklist must be completed before any production deployment:

- ☐ **Set `dial: 0`** — deploy in monitor-only mode
- ☐ **Pin image tags** — replace `latest` with a specific version
- ☐ **Change default passwords** — Grafana admin, Redis password
- ☐ **Secrets via Secret Manager** — no passwords in `.env` committed to

Table 68: Required firewall rules for DMZ deployment

Source	Destination	Port	Protocol	Purpose
Internet	HAProxy (DMZ)	443	TCP	Incoming TLS connections
HAProxy (DMZ)	JA4proxy (App)	8080	TCP	PROXY protocol + TLS passthrough
JA4proxy (App)	Redis (Data)	6379	TCP	State read/write
JA4proxy (App)	Backend (App)	443	TCP	Forwarded TLS connections
JA4proxy (App)	DNS resolver	53	UDP/TCP	FCrDNS lookups
JA4proxy (App)	AbuseIPDB API	443	TCP	Reputation lookup (outbound)
JA4proxy (App)	RDAP endpoints	443	TCP	RDAP lookup (outbound)
JA4proxy (App)	Spamhaus URLs	443	TCP	Blocklist refresh (outbound)
Prometheus (Mgmt)	JA4proxy (App)	9090	TCP	Metrics scraping
Loki (Mgmt)	JA4proxy (App)	(log push)	TCP	Log collection
Operator VPN	Grafana (Mgmt)	3001	TCP	Dashboard access
Operator VPN	Alertmanager (Mgmt)	9093	TCP	Alert management
Blocked (explicit DENY):				
Internet	JA4proxy (App)	any	any	Proxy never directly reachable from internet
Internet	Redis (Data)	any	any	Redis never internet-accessible
Internet	Mgmt zone	any	any	Management never internet-accessible

Git

- ☐ **TLS certificate for backend** — verify backend cert is valid and trusted
- ☐ **Trusted upstream CIDR** — configure HAProxy address range in proxy config
- ☐ **GeoIP database present** — MaxMind GeoLite2 files mounted and current
- ☐ **Redis persistence enabled** — RDB or AOF configured; data volume mounted
- ☐ **Loki log shipping** — proxy logs flowing to Loki; retention policy set
- ☐ **Prometheus scraping** — metrics endpoint reachable; dashboards loaded
- ☐ **Alert rules loaded** — minimum alert set from Chapter
- ☐ **HAProxy in TCP mode** — not HTTP mode; TLS passthrough verified
- ☐ **Management ports bound to 127.0.0.1** — not 0.0.0.0
- ☐ **Verify score distribution** — run for 24h at dial=0; review histogram
- ☐ **Phase 14 hardening** — complete Phase 14 checklist before raising dial above 0

Phase 14 Hardening Prerequisite

The Phase 14 production hardening checklist (`docs/DMZ_DEPLOYMENT_READINESS.md`) must be completed before raising the dial above 0 in production. Key gaps identified at POC stage include: secrets management integration, Redis TLS, rate-limit self-protection for the tarpit mechanism, and resource limit tuning. Do not skip this step.

Security Reference

JA4proxy is a security tool, but it is also a target. This chapter documents the security properties of JA4proxy itself: what it guarantees, where its boundaries are, what its known gaps are at POC stage, and how to harden it for production.

THE SECURITY MODEL OF JA4PROXY rests on three guarantees: it never decrypts traffic (cryptographic transparency), it fails open when uncertain (false positive minimisation), and it enforces a configurable hard floor below which browser traffic can never be blocked (ALPN structural bypass).

The Bypass System — Security Model

Bypass Architecture

Bypass conditions are not simply configuration flags — they represent a deliberate security policy decision about which classes of traffic should be exempt from scoring. Each bypass type has a specific threat model and risk profile when enabled or disabled.

Startup Warnings for High-Risk Bypass Disabling

When any bypass is disabled that has elevated false positive or false negative risk, JA4proxy emits a WARN level log at startup. This ensures the operator cannot accidentally disable a bypass in a config file and have the change go unnoticed.

```
# Example startup warnings for disabled bypasses
WARN | policy | event=bypass_disabled | bypass=alpn_browser_bypass
     | effect=browser traffic will be scored; false positive risk elevated

WARN | policy | event=bypass_disabled | bypass=spamhaus_drop_bypass
     | effect=Spamhaus DROP matches scored as signal(+80) instead of hard block
```

Table 69: Bypass security analysis — each bypass, its threat model, and risk of disabling

Bypass	Threat addressed	Risk if disabled	Use case for disabling
`alpn_browser_bypass`	Not applicable — this is a false-positive protection, not a threat bypass	Browser traffic scored; significant false positive risk at scale	Scoring specific h2 API clients known to not be real browsers
`ja4_whitelist_bypass`	Trusted fingerprints (known-good tools, monitoring) bypass scorer	Whitelisted fingerprints may be blocked if they score above threshold	Testing whether a whitelisted fingerprint would be scored
`mtls_bypass`	Cryptographic identity proofing — mTLS client is verified	mTLS clients scored and potentially blocked	Auditing mTLS client behaviour
`static_ip_allowlist`	Trusted internal IPs (monitoring, CI) bypass scorer	Monitoring may be blocked; CI tests fail	Temporary removal during security audit
`ja4_blacklist_bypass`	Known-bad fingerprints (C2, scanners) immediately rejected	Blacklisted fingerprints only blocked if score exceeds dial threshold	Investigating traffic from a specific fingerprint
`country_blacklist_bypass`	Geographically restricted access	Blocked countries go through scorer; may not be blocked at low dial	Investigating specific country traffic
`spamhaus_bypass`	Hijacked/criminal IP blocks immediately rejected	Spamhaus matches produce signal(+80) instead of hard block	Investigating Spamhaus-listed range
`tls_version_bypass`	Deprecated TLS versions immediately rejected	Old TLS produces risk signal instead of hard reject	Auditing legacy client population

The Asymmetry Principle — In Depth

Why Fail Open is Correct

The asymmetry principle (Chapter) has specific technical manifestations throughout the system. Understanding why each mechanism exists as it does requires understanding the cost model:

- **A false negative** (bad bot allowed) produces a malformed request to the backend. The backend logs it. The analytics node may detect the pattern retrospectively. The next connection from the same IP will have a better signal set (AbuseIPDB cache populated, beaconing window extended). Recovery is automatic.
- **A false positive** (real browser blocked) produces a failed page load for a human user. The user sees a connection error with no explanation. They may never return. There is no automatic recovery — the damage is immediate and irreversible.

False Positive Protection Mechanisms

These mechanisms exist specifically to prevent false positives:

ALPN structural bypass h2/h1 ALPN connections can never be blocked, regardless of score or dial setting. This is not a configurable threshold — it is structural. Even with dial=100 and all signals firing, a connection presenting h2 ALPN is allowed. Only if the operator explicitly disables this bypass does the structural protection lift.[5]

[5] Browser traffic presenting h2 or h1 ALPN could theoretically score highly if, for example, the IP is on Spamhaus DROP and the browser is behind a compromised ISP. The bypass ensures this never causes a false positive.

ALLOW cached 30min, BLOCK cached 30s The local LRU cache intentionally uses asymmetric TTLs. An IP allowed 29 minutes ago gets another 30-minute free pass. An IP blocked 31 seconds ago is re-evaluated against current Redis state.

Local cache wins on Redis failure When the local cache says allow and Redis says block (or Redis is unreachable), the local cache wins. Real users in the local cache continue to be served even during Redis outages.

External service unavailability → zero signal If AbuseIPDB is down, RDAP is unreachable, or DNS lookups time out, the affected signal contributes 0 to the score. The score can only go down when services fail, not up.

AbuseIPDB floor at 50 AbuseIPDB scores below 50 are suppressed. This prevents shared infrastructure (NAT gateways, corporate egress, VPNs) from triggering signals just because one user behind that NAT misbehaved.

RDAP block expansion off by default Expanding a block to an entire /24 based on one bad IP is a high-impact action. The default is off. Operators must consciously opt in.

The Fail-Open Guarantee

The fail-open guarantee is: **in all uncertainty, connections are allowed rather than blocked**. This guarantee applies to:

- Redis unreachable: bans not enforced; rate limits not enforced; signals that depend on Redis return 0
- External service (AbuseIPDB, RDAP, DNS) unreachable: affected signals return 0
- Config parse error on reload: previous valid config retained; new config not applied
- Pipeline exception: connection is allowed; exception is logged
- Unknown fingerprint: score 0 (unknown is not inherently suspicious)

Fail-Open is Not a Bug

Operators sometimes ask whether fail-open can be changed to fail-closed. The answer is no — this would be an architectural change that fundamentally compromises the false positive guarantee. Fail-closed means an outage blocks real users. The system is designed to tolerate outages gracefully, not to use them as an excuse to block.

Hot Config Reload — Security Implications

Hot config reload is a convenience, but it has security implications:

Policy change audit log Every config reload is written to `management:policy_audit` in Redis with a timestamp and attribution. Operators can audit all policy changes without consulting application logs.

Config file integrity The proxy does not verify the cryptographic integrity of `config/proxy.yml` before loading. In production, config files should be managed by a configuration management system (Ansible, Terraform, Kubernetes ConfigMap) with change control and audit trail.

SIGHUP race condition There is a brief window during config reload where new bypass states are being applied and some connections may be evaluated under the old configuration. This window is bounded by the duration of the reload operation (typically <50ms). No connections are dropped during reload.

Management UI reload The Management UI triggers reload via a Redis pub/sub message. If Redis is compromised, an attacker could trigger arbitrary config reloads. Redis authentication (`redis.password`) must be set in production.

Phase 200-Series Hardening

The Phase 200-series closes the production-hardening gaps flagged in Phase 14 and the comprehensive security audit. These items are now in production builds of the Go proxy.

Redis TLS (Phase 201)

Redis communications are now TLS-encrypted by default. The Go proxy connects to Redis with mutual TLS where the Redis instance is configured with `tls-port` and a server certificate; client certificates are loaded from the path configured under `redis.tls.client_cert_path`. Plaintext Redis connections remain available for development (a Unix-socket-only configuration with restrictive filesystem permissions is the recommended local-dev path), but production Helm charts default to TLS-on. See `docs/phases/PHASE_201.md` for the full configuration matrix.

PROXY Protocol v2 with TLVs (Phase 203)

The Go proxy now parses PROXY protocol v2 binary headers, including TLV (type-length-value) extensions. The TLV trust path is gated by `security_policy.trusted_upstream_cidrs`: a TLV is honoured only when the TCP peer is within the configured trusted CIDR. This prevents a direct-to-port-8080 attacker from forging TLVs to claim a whitelisted source IP. The trust check runs before any TLV is parsed, so a malformed TLV from an untrusted peer cannot trigger a parser path at all.

Default Credentials Removed (Phase 202)

Default credentials have been removed from the Docker image, Helm chart, and `docker-compose.yml`. Previously, Redis shipped with no `requirepass`, Grafana shipped with `admin/admin`, and the Management API had a hard-coded local-dev token. All three now require explicit env-var provisioning at startup; the proxy refuses to start if a required credential is unset. The Helm chart fails its preflight check and Docker Compose surfaces a clear error rather than silently starting with a blank password. See `docs/phases/PHASE_202.md` for the env-var reference and the migration runbook.

Known Security Gaps (POC Stage)

The following gaps are documented in `docs/DMZ_DEPLOYMENT_READINESS.md` and `docs/security/COMPREHENSIVE_SECURITY_AUDIT.md`. They must be addressed before production deployment with dial > 0.

Production Hardening Checklist

- ☐ **Secrets manager integration** — API keys and passwords from Vault or cloud secret manager; never in `.env` files
- ☐ **Redis TLS** — enable TLS for Redis connections or use Unix socket with restricted permissions
- ☐ **Redis AUTH** — set strong Redis password; verify it is not default
- ☐ **Redis ACLs** — separate credentials for proxy, analytics, and management services
- ☐ **Management UI authentication** — complete Phase 13 before deploying Management UI
- ☐ **Tarpit resource limits** — configure maximum concurrent tarpitted connections
- ☐ **Container non-root user** — verify proxy runs as non-root (uid 1000)
- ☐ **Read-only filesystem** — mount application directories read-only where possible

Table 70: Known security gaps at POC stage

Gap	Severity	Phase addressing it	Description
Secrets in environment variables	Medium	Phase 14	API keys and Redis password should come from a secret manager (Vault, AWS Secrets Manager), not plain environment variables
Redis without TLS	High	Phase 14	Redis communications are unencrypted. In production, Redis must use TLS or a Unix domain socket with filesystem permissions
Tarpit self-protection absent	Medium	Phase 14	The tarpit mechanism has no limit on concurrent tarpitted connections. A sustained flood could exhaust file descriptors
Config file not integrity-checked	Low	Phase 14	No signature verification on proxy.yml before hot-reload
Management UI authentication	High	Phase 13	Management UI has no authentication in POC. Must add before production
Redis ACLs not configured	Medium	Phase 34	All clients share a single Redis connection. Should use ACLs with per-service credentials
No Seccomp/AppArmor profiles	Low	Phase 55	Container syscall surface not restricted
Supply chain integrity	Low	Phase 35	No SBOM, no container image signing

- ☐ **Network policy** — enforce network zone isolation (see Chapter)
- ☐ **HAProxy trusted CIDR** — configure the exact HAProxy IP range in proxy trusted upstream config
- ☐ **Grafana admin password changed** — default `admin/admin` must not be used
- ☐ **Loki log retention** — configure retention policy (30 days recommended for GDPR compliance)
- ☐ **Audit log retention** — verify `management:policy_audit` retention policy
- ☐ **Pentest validation** — complete the adversarial test suite (Phase 45) before raising dial

IP Spoofing Protection

Without trusted upstream CIDR verification, an attacker who can reach port 8080 directly could inject a PROXY protocol header claiming any source IP address — effectively bypassing all IP-based checks by claiming to be a whitelisted IP.

JA4proxy mitigates this by verifying that the sender of the TCP connection (before reading the PROXY protocol header) is within the configured trusted upstream CIDR. Only HAProxy addresses should be in this CIDR. If the sender is not in the trusted CIDR, the PROXY protocol header is rejected and the real TCP peer IP is used instead.

> **Port 8080 Must Not Be Internet-Accessible**
>
> Port 8080 (the JA4proxy listen port) must only be reachable from HAProxy. If an attacker can reach port 8080 directly, they can manipulate their apparent source IP. Firewall rules must block port 8080 from all sources except the HAProxy IP range.

JA4 Fingerprint Limitations

JA4 fingerprinting is powerful but not infallible. Security engineers should understand its limitations:

JA4 copying An attacker can copy a known-good JA4 fingerprint by configuring their TLS library to match the exact cipher suite list and extension ordering. JA4T (TCP fingerprint) makes this harder — it requires matching the OS-level TCP stack, not just the TLS library.

JA4 rotation A sophisticated attacker can rotate through JA4 values to evade per-fingerprint rate limits. The `by_ip_ja4_pair` rate limit strategy is designed to catch this pattern.

JA4 collisions Multiple legitimate clients may share the same JA4 value. JA4 blacklisting should only be applied to fingerprints that are *uniquely* associated with malicious tools — not fingerprints shared with any legitimate client software.

Compliance Reference

JA4proxy processes IP addresses and TLS fingerprints in the course of its security function. This chapter documents what personal data is processed, for how long, what controls apply, and how to satisfy data subject requests.

Compliance obligation depends on deployment context. JA4proxy is a security proxy; the operator who deploys it is the data controller. This chapter provides the technical facts that a data controller needs to assess their compliance obligations. It does not constitute legal advice.

Data Inventory

What Data is Processed

Table 71: Data elements processed by JA4proxy

Data element	Personal data?	Where stored
Client IP address (IPv4/IPv6)	Yes (see note)	Redis, logs, Prometheus labels
JA4 fingerprint	No (device, not person)	Redis, logs
JA4X fingerprint	No	Redis, logs
JA4T fingerprint	No	Redis, logs
SNI hostname	No (operator's domain)	Logs
Country code	No (GeoIP; not precise location)	Redis, logs
ASN organisation name	No	Logs (transient)
AbuseIPDB score	Derivative of IP (see note)	Redis (cache)
RDAP organisation	No	Redis (cache)
PTR/FCrDNS hostname	Potentially (if ISP assigns personal hostnames)	In-process cache (transient)
Beaconing timestamps	Linked to IP (see note)	Redis Sorted Set
Connection metadata (port, timing)	Potentially linked to IP	Logs, Prometheus

IP Addresses as Personal Data

Under GDPR Recital 30, IP addresses are personal data when they can be linked to a natural person. Dynamic IP addresses assigned by ISPs to residential customers are generally considered personal data in the EU. Static IP addresses assigned to organisations are generally not personal data. JA4proxy processes both categories and cannot distinguish them at the time of processing. Operators must account for this in their DPIA.

Legal Basis for Processing

The operator must establish a legal basis for processing IP addresses. Typical bases:

Legitimate interests (Art. 6(1)(f)) Security monitoring and DDoS/bot mitigation is a legitimate interest of the controller. The processing is proportionate because: only metadata is processed (no content decryption), data is retained for the minimum period necessary, and the security purpose is genuine.

Legal obligation (Art. 6(1)(c)) If the operator is subject to regulations requiring security controls (PCI-DSS, NIS2), processing for compliance purposes has a legal basis.

Data Retention

Redis Key TTLs

Table 72: Redis key TTLs for compliance purposes

Key type	Contains IP?	TTL	Notes
`ban:{ip}`	Yes	300s–604800s	Escalating ban. Max 7 days.
`rate:{strategy}:{key}`	Yes (in key)	60s window	Sliding window; entries expire with w
`beacon:{ip}:{ja4}`	Yes (in key)	30min window	Timestamps expire; key persists until
`conn:{ip}`	Yes (in key)	Connection lifetime	Deleted when connection count reach
`visit:{ip}`	Yes (in key)	None (persistent)	Manual deletion required for erasure
`sess:{ip}`	Yes (in key)	None (persistent)	Manual deletion required for erasure
`hll:cidr24:{cidr}`	Yes (CIDR containing IP)	30 days	Approximate; cannot extract individu
`abuseipdb:{ip}`	Yes (in key)	3600s	Cache; auto-expires
`geo:{ip}`	Yes (in key)	86400s	Cache; auto-expires
`rdap:{ip}`	Yes (in key)	86400s	Cache; auto-expires
`findings:{ip}`	Yes (in key)	300s	Analytics output; auto-expires
`management:policy_audit`	No	None (max 1000 entries)	Audit log; no IP data

Log Retention

Connection logs are emitted to stdout and shipped to Loki (or another log aggregator). The retention period is configured in the log aggregation system:

- **Recommended retention:** 30 days for operational logs
- **Security incident retention:** 90 days (check applicable law)
- **Legal hold:** Operator-specific; coordinate with legal team

Log retention is configured in Loki's `storage_config` section. JA4proxy does not enforce log retention directly — this is the responsibility of the log aggregation pipeline.

Prometheus Metrics Retention

Prometheus metrics contain IP addresses only in specific label combinations (notably the ban metrics, which label by IP). Default Prometheus retention is 15 days. For compliance, either:

1. Reduce retention to 7 days for metrics containing IP labels
2. Configure remote write to a long-term storage system with access controls
3. Consider using IP prefix labels (/24 or /48) instead of full IPs for metrics

Right to Erasure (GDPR Art. 17)

Data Subject Access Request (DSAR) Process

When a data subject submits a right-to-erasure request identifying their IP address, the operator must delete all personal data associated with that IP. The following Redis keys must be deleted:

```
# DSAR erasure script for a specific IP address
# Replace TARGET_IP with the subject's IP address

TARGET_IP="203.0.113.42"

# Delete all Redis keys containing this IP
redis-cli DEL "ban:${TARGET_IP}"
redis-cli DEL "conn:${TARGET_IP}"
redis-cli DEL "visit:${TARGET_IP}"
redis-cli DEL "sess:${TARGET_IP}"
redis-cli DEL "abuseipdb:${TARGET_IP}"
redis-cli DEL "geo:${TARGET_IP}"
redis-cli DEL "rdap:${TARGET_IP}"
redis-cli DEL "findings:${TARGET_IP}"

# Delete beaconing keys (need to enumerate JA4 variants)
redis-cli SCAN 0 MATCH "beacon:${TARGET_IP}:*" COUNT 100
# Delete each matched key

# Delete rate limit keys
redis-cli SCAN 0 MATCH "rate:*:${TARGET_IP}*" COUNT 100
# Delete each matched key

# Logs: use log aggregation platform API to delete/redact log
    entries
# HyperLogLog: individual IPs cannot be removed from HLL -
    document this limitation
```

HyperLogLog Erasure Limitation

HyperLogLog data structures (`hll:cidr24:` and `hll:cidr48:`) do not support removal of individual elements. Once an IP has been added to a HLL, it cannot be removed without deleting the entire HLL key (which would also remove all other IPs from that CIDR). Document this technical limitation in your DPIA and assess whether HLL data constitutes personal data under your jurisdiction (it does not reveal the actual IP address, only an approximate count).

IP Anonymisation Option

For environments where storing full IP addresses is not appropriate, JA4proxy can be configured to truncate IP addresses in logs and Redis keys:

- IPv4: truncate last octet (e.g. `203.0.113.42` → `203.0.113.0`)
- IPv6: truncate to /48 (e.g. `2001:db8::1` → `2001:db8::`)

IP Anonymisation Degrades Security Function

Truncating IP addresses prevents ban enforcement (you cannot ban `203.0.113.0/24` without blocking 255 potentially innocent IPs). It also prevents return visitor tracking, session resumption ratio, and concurrent connection counting. IP anonymisation severely degrades the security function of JA4proxy and should only be used when legally required, with operator acknowledgement of the security trade-off.

PCI-DSS Considerations

JA4proxy contributes to the following PCI-DSS v4.0 requirements when deployed in front of a cardholder data environment:

Table 73: PCI-DSS v4.0 control contributions

Requirement	Control	JA4proxy contribution
1.2	Network access controls	Enforces perimeter acces try, and TLS fingerprint
1.3	Network access restriction	Blocks traffic from kno (Spamhaus, blacklisted
6.4	WAF/bot protection	TLS fingerprint analys tools and scanners
6.5	Change management	Config audit log (`man` provides change trail
10.2	Audit logs	Structured JSON logs c with action, IP, and reas
10.3	Audit log protection	Logs shipped to Loki; p own logs
10.5	Log retention	Log retention policy (se
12.10	Incident response | Security monitoring via anomaly detection and alerting	

SOC 2 Audit Trail Completeness

For SOC 2 Type II assessments, auditors typically request evidence of:

Access control decisions Every connection produces a JSON log entry with action, reason, and client identification. The structured log format provides a complete, queryable audit trail.

Configuration change management The `management:policy_audit` Redis list records every policy change with timestamp, operator attribution, and old/new values. This list is limited to 1000 entries — for long-term retention, ship audit log entries to an external store.

Security monitoring Prometheus metrics, Grafana dashboards, and Alertmanager alert rules provide evidence of continuous security monitoring.

Incident response The analytics node's campaign detection and score drift alerting provide evidence of automated anomaly detection.

GDPR Data Flow Diagram

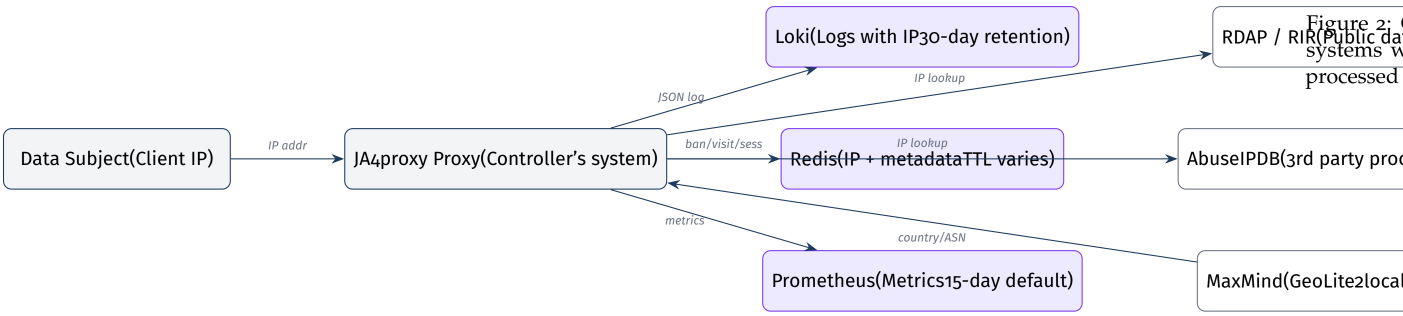

Figure 2:
systems w
processed

Third-Party Data Processors

JA4proxy sends client IP addresses to the following third-party services. Operators must ensure appropriate data processing agreements (DPAs) are in place:

Table 74: Third-party data processors

Service	Data sent	Retention by processor	DPA required?
AbuseIPDB	Client IP address	Per AbuseIPDB privacy policy	Yes, if processing EU sonal data
RDAP/WHOIS (RIRs)	Client IP address	Public records; varies by RIR	Assess per RIR
MaxMind GeoLite2	None (local DB)	N/A — local file, no network calls	No
DNS resolver	Reverse DNS query for client IP	Per resolver retention policy	Depends on resolver

Disabling AbuseIPDB for Privacy Compliance

If your jurisdiction or DPA requires that client IP addresses not be sent to third parties without consent, set `abuseipdb.enabled: false`. This removes the only signal that requires active transmission of client IPs to an external service. RDAP queries can also be disabled (`rdap.enabled: false`). MaxMind GeoLite2 operates entirely on a local database file with no network calls.

Glossary

Terms are listed alphabetically. Cross-references point to the chapter or section where each concept is covered in detail.

Action decider
The final pipeline layer (Layer 14) that maps a computed risk score and the current dial setting to one of the six actions: allow, flag, rate_limit, tarpit, block, or ban. The action decider applies the operator's dial policy to the objective score produced by the risk scorer. See Chapter , Section on the Action Decider.

ALPN (Application-Layer Protocol Negotiation)
A TLS extension (extension type 0x0010) in the ClientHello that lists the application-layer protocols the client supports, in preference order. The most common values for HTTPS are `h2` (HTTP/2) and `h1` (HTTP/1.1). JA4proxy uses ALPN to identify browser traffic — real browsers always negotiate h2 or h1. See Chapter , Layer 1.

ASN (Autonomous System Number)
A globally unique number assigned to a network operator (ISP, cloud provider, university, etc.) by a Regional Internet Registry (RIR). IP prefixes are announced under an ASN in the global BGP routing table. JA4proxy uses the MaxMind GeoLite2 ASN database to map client IPs to their originating ASN and organisation name. See Chapter , Signal 4.

AbuseIPDB
A community-maintained reputation database of IP addresses reported for malicious activity (scanning, brute-force, spam, C2 traffic). Each IP receives a confidence score 0–100. JA4proxy caches AbuseIPDB responses in Redis and uses the score as a weighted signal. Scores below 50 are suppressed to avoid false positives from shared infrastructure. See Chapter , Signal 7.

Attacker attribution
Pipeline Layer 11. The process of correlating JA4, JA4X, and JA4T fingerprints across multiple source IPs to identify coordinated campaigns, botnet activations, and IP-rotation evasion attempts. Attribu-

tion findings are written by the analytics node and read back by the pipeline. See Chapter , Section on Layer 11.

Asymmetry principle
The core design principle of JA4proxy: the cost of a false positive (real user blocked) is higher than the cost of a false negative (bad bot allowed). This principle governs all threshold defaults, caching TTLs, and fallback behaviours. When in doubt, fail open. See Chapter , Section on Design Principles.

Ban TTL
The time-to-live of an IP ban entry in Redis (`ban:{ip}`). Initial TTL is configurable (default 300 seconds); it escalates by doubling on each re-ban up to a maximum of 604800 seconds (7 days). After the TTL expires, the ban entry is automatically removed by Redis and the IP is re-evaluated on next connection. See Chapter , Section on Blocking Escalation.

Beaconing
A network traffic pattern characterised by regular, periodic connections with low variance in inter-arrival times (IAT). Beaconing is characteristic of C2 (command and control) implants and automated scheduled tasks, as opposed to human browsing which has highly variable timing. Detected using the coefficient of variation of IAT. See Chapter , Signal 6.

Bloom filter
A probabilistic data structure that supports membership queries with a controlled false positive rate and no false negatives. JA4proxy uses Redis Bloom filters (via RedisBloom module) for enrichment deduplication — to avoid re-querying AbuseIPDB or RDAP for IPs that have already been processed. If RedisBloom is unavailable, a Redis SET with a 24h TTL is used as fallback. See Chapter .

Bypass condition
A fast-path rule that short-circuits the pipeline immediately when a specific condition is met, without running the signal collection or scoring layers. JA4proxy has eight bypass conditions, each independently configurable. See Chapter , Section on the Fast-Path Bypass System.

CIDR (Classless Inter-Domain Routing)
IP prefix notation combining a base address with a prefix length (e.g. `185.220.96.0/21`). Used in JA4proxy for subnet blocking, trusted upstream CIDR verification, and block expansion. CIDR matching is always performed in-process using a trie data structure — never via Redis queries. See Chapter .

ClientHello
The first message sent by a TLS client in a TLS handshake. It is transmitted in plaintext (TLS record type 0x16, content type 0x01) before any encryption is established. It contains the TLS version, cipher suite list, extensions (including SNI, ALPN, supported groups), and other parameters. JA4proxy reads and parses the ClientHello without participating in the TLS handshake. See Chapter , Section on Data Flow.

Cold cache
The state of the local in-process LRU cache immediately after startup, or for a newly seen IP address. Cold cache state means there is no locally cached decision; the pipeline must execute in full (including Redis lookups and signal collection) to produce a decision for this IP. Contrast with *warm cache*.

Coefficient of variation (CV)
A statistical measure of dispersion: CV = standard deviation / mean. Used in the beaconing detector to quantify the regularity of inter-arrival times. Low CV (<0.4) indicates regular, beaconing-like timing. High CV (>0.8) indicates variable, human-like timing. See Chapter , Signal 6.

Dial
A scalar integer setting (0–100) that controls how aggressively risk scores are translated into blocking actions. At dial=0, the proxy is in monitor-only mode — all connections are allowed regardless of score. At dial=100, configured thresholds apply fully. The dial should be raised incrementally after validating the score distribution. See Chapter , Section on dial.

DGA (Domain Generation Algorithm)
An algorithm used by malware to generate pseudo-random domain names for C2 communication, making blacklisting difficult. DGA-generated domains typically have high Shannon entropy and use unusual TLD patterns. JA4proxy detects DGA-like SNI values as a scored signal. See Chapter , Signal 2.

FCrDNS (Forward-confirmed reverse DNS)
A DNS validation technique: the PTR record for an IP resolves to a hostname, and that hostname's A/AAAA record resolves back to the original IP. FCrDNS is a lightweight indicator of network infrastructure legitimacy — residential ISPs, major cloud providers, and CDNs maintain consistent FCrDNS. See Chapter , Signal 5.

Fail open
The principle that in all uncertainty or failure conditions, connections are allowed rather than blocked. An unavailable external service,

a Redis error, or an unrecognised fingerprint all result in a zero signal contribution — the score can only decrease when services fail, not increase. The fail-open guarantee is fundamental to JA4proxy's false positive minimisation. See Chapter , Section on the Fail-Open Guarantee.

Fast path
The set of pipeline layers (0–2) that short-circuit immediately without running the signal collection or scoring layers. Fast-path decisions (from local cache, bypass conditions) complete in under 100 microseconds. Connections that bypass the fast path proceed to the signal collection zone. See Chapter .

HyperLogLog
A probabilistic data structure for counting distinct elements with fixed memory cost (12KB per key in Redis). Used in JA4proxy to count unique IPs per /24 (IPv4) or /48 (IPv6) CIDR prefix. Standard error is 0.81%. Individual elements cannot be removed from a HyperLogLog without clearing the entire structure. See Chapter .

IAT (Inter-arrival time)
The time elapsed between consecutive connections from the same source (IP + JA4 fingerprint pair). IAT analysis is the core of the beaconing detection algorithm. IAT values are stored as timestamps in a Redis Sorted Set with a 30-minute sliding window. See Chapter , Signal 6.

JA4
A TLS client fingerprinting algorithm that produces a deterministic hash from the TLS ClientHello: cipher suites (sorted), TLS extensions (sorted), ALPN values, and signature algorithms. Produces a string of the form `t13d1516h2_8daaf6152771_02713d6af862`. The algorithm was developed by FoxIO and is defined at `https://github.com/FoxIO-LLC/ja4`. See Chapter .

JA4X
The extended JA4 fingerprint that includes the TLS extension type and ordering information — not just which extensions are present, but in what order the client sent them. JA4X can distinguish clients that share the same JA4 value but have different extension ordering (e.g., a real Chrome vs a JA4-copying tool). See Chapter .

JA4T
The TCP transport fingerprint component of the JA4 family. Encodes the TCP window size, TCP options (MSS, window scale, SACK, timestamps), and their ordering. JA4T fingerprints the OS-level TCP/IP stack, making it harder to spoof than JA4 (which fingerprints the TLS library layer). See Chapter , Signal 3.

Monitor mode
The operating mode when `dial: 0`. In monitor mode, the full pipeline runs (all signals collected, scores computed, actions decided), but every connection is allowed regardless of score. Actions are logged as MONITOR. Monitor mode is the default and recommended starting point for all deployments. See Chapter , Section on the Action Decider.

mTLS (Mutual TLS)
TLS authentication where both the server and the client present certificates. A valid mTLS client certificate is cryptographic proof of identity — the client holds the private key corresponding to a certificate signed by the configured trusted CA (`config/trusted_cas.pem`). By default, mTLS clients bypass the scoring pipeline entirely. See Chapter , Layer 1c.

PROXY protocol
A binary or text framing format prepended to a TCP stream by a load balancer to convey the original client IP address and port. JA4proxy requires HAProxy to use PROXY protocol v2 (binary format) so that the real client IP is available after the TCP connection arrives from HAProxy. See Chapter , Section on Stage 1.

RDAP (Registration Data Access Protocol)
The modern replacement for WHOIS. Provides structured JSON responses about IP address registrations from Regional Internet Registries (RIRs: ARIN, RIPE, APNIC, LACNIC, AFRINIC). JA4proxy uses RDAP to identify the organisation owning an IP block and cross-reference it against the known-bad organisations list. See Chapter , Signal 8.

Risk score
A scalar integer in the range 0–100 produced by the risk scorer (pipeline Layer 13) by confidence-weighted aggregation of all signal contributions. A score of 0 means clean; 100 means maximum risk. The dial setting controls how the score is translated into a blocking action. See Chapter , Section on the Risk Scorer.

Signal
A scored observation produced by one of the eight signal checks (pipeline Layers 3–10). Each signal returns a `RiskSignal(score, weight, reason)` object. Signals run concurrently via `asyncio.gather()`. A signal that cannot produce a result (service unavailable) returns score=0 (fail open). See Chapter .

SNI (Server Name Indication)
A TLS extension (type 0x0000) in the ClientHello that specifies the hostname the client intends to connect to. SNI allows a single server

to host multiple TLS virtual hosts on the same IP. JA4proxy uses SNI to detect missing SNI, IP-literal SNI, and DGA-like domain names. See Chapter , Signal 2.

Spamhaus DROP/EDROP
The Spamhaus Don't Route Or Peer (DROP) and Extended DROP (EDROP) blocklists contain IP ranges that have been hijacked or directly allocated to cybercriminals. False positive rate is near zero. JA4proxy downloads these lists at startup and on a refresh interval, storing them in an in-process trie for O(log n) lookup. See Chapter , Layer 0e.

Tarpit
An action that accepts a connection but delivers data to the backend at an artificially throttled rate, consuming the attacker's connection slots and CPU while expending minimal resources on the defender's side. Effective against connection-limited scanners and brute-force tools. JA4proxy implements tarpit as a rate-controlled TCP relay. See Chapter , Section on the Six Actions.

TLS fingerprinting
The technique of computing a deterministic identifier from the parameters in a TLS ClientHello (cipher suites, extensions, ALPN, etc.) to identify the TLS client implementation. Different applications produce different fingerprints even when connecting to the same server. The JA4 algorithm is the fingerprinting standard used by JA4proxy. See Chapter .

Warm cache
The state of the local in-process LRU cache when it contains a recent decision for a given IP address. A warm cache hit means the pipeline can return a decision in under 100 microseconds without any Redis or signal collection overhead. Cache entries have asymmetric TTLs: ALLOW entries are cached for 30 minutes; BLOCK entries for 30 seconds. Contrast with *cold cache*.

Index

JA4proxy

Enterprise TLS Security Proxy

What JA4proxy does

JA4proxy intercepts the TLS ClientHello — which is always transmitted in plaintext, before encryption begins. It computes a JA4 fingerprint from the cipher suites, extensions, ALPN, and elliptic curves advertised by each client, then routes the connection through a multi-signal scoring pipeline to decide: *allow, tarpit, block, or ban.* Allowed connections are forwarded byte-for-byte unchanged. The proxy never decrypts, never holds your private keys, and is transparent to both client and backend.

Key facts

- 0% false positive rate on browser traffic — by design
- 94–99% of malicious traffic blocked in testing
- Default dial = 0: monitor mode, never blocks on first deploy
- All bans carry a 5-minute TTL — false positives self-heal
- Never decrypts: no key custody, no SSL inspection compliance risk
- Scales horizontally; Redis synchronises bans across all instances

Resources

Source code & issue tracker: `github.com/seanpor/JA4proxy`

JA4 fingerprint specification: `github.com/FoxIO-LLC/ja4`

Documentation index: `docs/README.md` in the repository